

Projekt: Krasnal

Eryk Daczyński
Małgorzata Maćkowiak
Jeremiasz Waclawski
Aleks Zieliński

May 29, 2026

Contents

1	Informacje wstępne	4
1.1	Treść projektu	4
1.2	temp	5
2	Funkcje	5
2.1	det()	5
2.2	EPS.h	6
2.2.1	cmpDouble()	6
2.2.2	sgn()	6
2.2.3	Kod źródłowy	6
2.3	is_point_on_segment()	6
2.4	do_segments_intersect()	7
2.5	distance()	7
2.6	is_point_in_polygon()	7
2.7	graham_scan()	8
2.8	KMP()	9
3	Struktury	10
3.1	Point	10
3.2	Huffman.cpp	11
3.2.1	HuffmanNode	12
3.2.2	HuffmanCode	12
3.2.3	buildHuffmanTree()	12
3.2.4	generateHuffmanCodes()	12
3.2.5	Huffman()	13
3.2.6	HuffmanPrint()	13
3.2.7	HuffmanEquals()	13
3.2.8	HuffmanSort()	13
3.2.9	Kod źródłowy	13
3.3	Edge.h	15
3.4	Graph.cpp	15
3.5	Kopalnia.cpp	15
3.6	Krasnal.cpp	17
3.7	MinCostMaxFlow.cpp	18
3.8	SegmentTree.cpp	20
3.8.1	build()	20
3.8.2	query()	20
3.8.3	update()	20
3.9	Straznik.cpp	21
4	Solvery	22
4.1	BorderPatrolSolver	22
4.1.1	calculateConvexHull()	22
4.1.2	calculatePatrolDistance()	22
4.1.3	Kod źródłowy	22
4.2	WorkAssignmentSolver	23
4.2.1	Model kosztu	23
4.2.2	Wynik	23
4.2.3	Kod źródłowy	23
4.3	GuardCommandSolver	28
4.3.1	Rozstrzygnięcie remisów	28
4.3.2	Aktualizacja	28
4.3.3	Kod źródłowy	28

5	Testy funkcji	30
5.1	test_det.cpp	30
5.2	test_is_point_on_segment.cpp	32
5.3	test_do_segments_intersect.cpp	32
5.4	test_is_point_in_polygon.cpp	33
5.5	test_graham_scan.cpp	34
6	Testy struktur	37
6.1	test_point.cpp	37
6.2	test_huffman.cpp	38
6.3	test_segment_tree	40
6.4	test_class_BorderPatrol.cpp	41
6.5	test_work_assignment_solver.cpp	43
6.6	test_segment_tree.cpp	44
6.7	test_guard_command_solver.cpp	45
7	Prezentacja działania	47
7.1	Główny przepływ programu	47
7.2	Przykładowy wynik dla Dane/dane.txt	47

1 Informacje wstępne

1.1 Treść projektu

Pracując nad poniższym projektem, należy: odfiltrować informacje istotne od nieistotnych sformalizować problem i podać jego specyfikację; do rozwiązania wykorzystać odpowiednie struktury danych oraz algorytmy; opisać teoretycznie rozwiązanie; przeanalizować jego optymalność oraz złożoność czasową/pamięciową; zaimplementować rozwiązanie w dowolnym języku programowania; przeprowadzić testy na różnych przykładach.

Problem

Odkąd królowa Śnieżka wraz z księciem z bajki rządzą krasnoludkami, ich królestwo nabiera siły i rośnie. Obecnie zamieszkuje je o wiele więcej niż tylko siedmiu krasnoludków. Każdy z nich jest specjalistą w swoim fachu i w kopalniach lub wyrobiskach potrafi wydobywać różne minerały. Królowa zna każdego ze swoich poddanych i wie jakie minerały każdy z nich potrafi wydobywać. To ona decyduje o miejscu pracy każdego z krasnoludków, jest to dla niej niełatwym zadaniem. Chcąc dbać o swoich obywateli, zapewniając im dostatek, królowa zleciła badania geologiczne w swoim kraju. Dzięki temu ma informacje o położeniu poszczególnych złóż w swoim kraju i wie, jaka jest ich wydajność (ilu krasnoludków może pracować w tym miejscu). Okazuje się, że najważniejsza jest miłość do pracy, którą się wykonuje. To znaczy, że jeśli krasnoludek pracuje przy materii zgodnej z jego preferencjami, wydobędzie kruszec o stałej wartości, niezależnie czy kopie złoto, węgiel, rudę miedzi, czy inny surowiec. Królowa Śnieżka poprosiła, abyś na podstawie posiadanych przez nią informacji pomógł jej przydzielić pracę każdemu z krasnoludków.

Po rozmowie z księciem okazało się, że jego ukochana bardzo dużo czasu spędza w kuchni, nadzorując gotowanie owsianki dla wygłodniałych brodaczy. Ilość potrzebnej owsianki zwiększa się, gdy krasnoludki muszą pokonać większą odległość z domu do pracy (można przyjąć, że jest to jedyny czynnik). Książę, chcąc ulżyć swojej wybrance, zmierzył i spisał odległości, pomiędzy każdym z wyrobisk i domkiem każdego z krasnoludków. Prosi, abyś uwzględnił te informacje. Przy podziale pracy chciałby, aby sumaryczna odległość, którą każdego dnia muszą pokonać krasnoludki była możliwie jak najmniejsza, przy czym nie możemy pozwolić, żeby spadła wartość produkowanych dóbr.

Niestety nie jest to jedyny problem. Królowa Śnieżka po incydencie ze złą królową jest nieco drażliwa, gdy pojawia się temat jabłek. Nakazała wyciąć wszystkie jabłonie w królestwie i zabroniła spożywania owoców. Krnąbrne krasnoludki starają się jednak wzbogacić swoją dietę o pyszne renety, antonówki, cortlandy, papierówki i ligole. Szmuglują przez granice królestwa pewne ilości tych owoców. W celu egzekucji prawa, zakazującego spożywania jabłek na terenie królestwa, książę codziennie rano musi okrążyć teren, na którym znajdują się wszystkie użytkowane kopalnie. Chciałby, żeby ta codziennie pokonywana odległość była możliwie mała. Gdy i to nie przynosiło skutku, książę wzdłuż trasy swego przejazdu rozstawił dekametrowców (krasnoludki uzbrojone w łuki ustawione co określoną liczbę metrów). Mają one zapobiec przerzucaniu jabłek przez granicę. Ataki takie wciąż zdarzają się często, atakowany jest dany wielometrowy odcinek granicy, przez który leci niezliczona liczba jabłek w jednym momencie. Dekametrowcy nie mogą opuścić swojego odcinka granicy, a najskuteczniejszą obroną przed lecącymi jabłkami okazuje się salwa.

Najgłośniejszy z krasnoludków atakowanej części granicy powinien wydać sekwencję poleceń:

„strzały na cięciwy
- naciągnąć cięciwy
- strzał!”.

Głos krasnoludka jest jego dumą i sprawą drażliwą, niełatwo więc przychodzi im ustalenie najgłośniejszego. Zaproponuj szybki sposób znajdowania dekametrowca z danego kawałka granicy, który winien wydać rozkazy.

Jak widać, królowa i książę muszą wykonać ogrom pracy, wykorzystując skomplikowane działania. Martwią się, że ta wiedza zaginie. Pomóż im zapisać cenną wiedzę w elektronicznych księgach, oszczędzając elektroniczny papier. Zatroszcz się, żeby potomni mogli szybko

wyszukiwać potrzebne informacje.

Co powinny przygotować zespoły studenckie

W ramach projektu należy przygotować rozwiązanie problemu przedstawionego powyżej.

Rozwiązanie powinno składać się z:

- dokumentacji
- plików programów
- opisu poprawności rozwiązania
- raportów przeprowadzonych testów

Wyniki pracy powinny być zapisywane z wykorzystaniem systemu Git (prowadzący powinien mieć dostęp do repozytorium). Projekt będzie oceniany zgodnie z informacjami przedstawionymi na USOS. Każda osoba będzie musiała wypełnić dwie ankiety: opisującą jej wkład w rozwiązanie problemu oraz oceniającą kompletność rozwiązania problemów przez zespół, do którego należy.

1.2 temp

```
1 int main(){
2     int i = 0;
3
4     return i;
5 }
```

Plik źródłowy 1: x example

2 Funkcje

2.1 det()

Funkcja pozwalająca ustalić położenie punktu p_3 względem wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) > 0$, to p_3 leży po lewej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) < 0$, to p_3 leży po prawej stronie wektora $\overrightarrow{p_1p_2}$

Jeżeli $\det(p_1, p_2, p_3) = 0$, to punkty p_1, p_2, p_3 są współliniowe

```
1 #include "det.h"
2
3 double det(Point p1, Point p2, Point p3) {
4     // 3 x 3
5     // D = (x2 - x1)(y3 - y1) - (y2 - y1)(x3 - x1)
6
7     /*
8         | 1 x1 y1 | = p1
9         | 1 x2 y2 | = p2
10        | 1 x3 y3 | = p3
11
12        D > 0 => p3 na lewo od p1p2
13        D < 0 => p3 na prawo od p1p2
14        D = 0 => p3 "na" p1p2 (współliniowy)
15
16        */
17
18     return (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x);
19 }
```

Plik źródłowy 2: /Struktury/Funkcje/det.cpp

2.2 EPS.h

2.2.1 cmpDouble()

cmpDouble(a, b) sprawdza która liczba jest większa z uwzględnieniem błędu typu liczbowego double

2.2.2 sgn()

sgn(x) sprawdza znak liczby *x*, korzystając z *cmpDouble(x, 0.0)* [dokładniej, to sprawdza czy liczba jest większa od 0]

2.2.3 Kod źródłowy

```
1 #ifndef EPS_H
2 #define EPS_H
3
4 constexpr double EPS = 1e-9;
5
6 inline int cmpDouble(double a, double b) {
7     if (a < b - EPS) return -1;
8     if (a > b + EPS) return 1;
9     return 0;
10 }
11
12 inline int sgn(double x) {
13     return cmpDouble(x, 0.0);
14 }
15
16 #endif
```

Plik źródłowy 3: /Struktury/Funkcje/EPS.h

2.3 is_point_on_segment()

W tej funkcji sprawdzamy przynależność punktu *P* do odcinka p_1p_2

Pierw sprawdzamy czy punkt *P* jest współliniowy z $\overrightarrow{p_1p_2}$, jeżeli nie to nie należy do odcinka p_1p_2

Następnie jeżeli jest współliniowy, to sprawdzamy kolejno współrzędne *x* i *y* czy spełniają warunki odpowiednio:

$$\min(p_1.x, p_2.x) \leq P.x \leq \max(p_1.x, p_2.x)$$

$$\min(p_1.y, p_2.y) \leq P.y \leq \max(p_1.y, p_2.y)$$

Jeżeli spełnione są oba te warunki to punkt *P* należy do odcinka p_1p_2

```
1 #include "is_point_on_segment.h"
2
3 // Sprawdzenie czy punkt P należy do odcinka p1p2
4 bool is_point_on_segment(Point p1, Point p2, Point P)
5 {
6     // P należy gdy p1 p2 i P sa wspolliniowe -> D = 0
7     if (sgn(det(p1, p2, P)) != 0){
8         return false;
9     }
10    // P należy gdy min(x1,x2) <= x3 <= max(x1,x2)
11    // oraz max(y1, y2) <= y3 <= max(y1, y2)
12    bool x_ok = cmpDouble(std::min(p1.x, p2.x), P.x) <= 0 &&
13                cmpDouble(P.x, std::max(p1.x, p2.x)) <= 0;
14    bool y_ok = cmpDouble(std::min(p1.y, p2.y), P.y) <= 0 &&
15                cmpDouble(P.y, std::max(p1.y, p2.y)) <= 0;
16
17    return x_ok && y_ok;
18 }
```

Plik źródłowy 4: /Struktury/Funkcje/is_point_on_segment.cpp

2.4 do_segments_intersect()

Funkcja sprawdza czy odcinki p_1p_2 i p_3p_4 przecinają się

Obliczamy $D_1 = \det(p_1, p_2, p_3)$, $D_2 = \det(p_1, p_2, p_4)$, $D_3 = \det(p_3, p_4, p_1)$ i $D_4 = \det(p_3, p_4, p_2)$

Jeżeli $D_1 \cdot D_2 < 0$ i $D_3 \cdot D_4 < 0$ to p_1p_2 i p_3p_4 przecinają się

Jeżeli $D_1 = 0$ lub $D_2 = 0$ lub $D_3 = 0$ lub $D_4 = 0$ to sprawdzamy przynależności punktu do prostej (ponieważ $\overrightarrow{p_1p_2}$ i $\overrightarrow{p_3p_4}$ są współliniowe)

```
1 #include "do_segments_intersect.h"
2
3 // Sprawdzenie czy odcinki p1p2 p3p4 sie przecinaja
4 bool do_segments_intersect(Point p1, Point p2, Point p3, Point p4)
5 {
6     double d1 = det(p1,p2,p3);
7     double d2 = det(p1,p2,p4);
8     double d3 = det(p3,p4,p1);
9     double d4 = det(p3,p4,p2);
10
11     // Przecinaja sie gdy:
12     // WARUNEK 2: kazdy odcinek przecina prosta wyznaczona przez drugi odcinek
13     if((sgn(d1) * sgn(d2) < 0) && (sgn(d3) * sgn(d4) < 0)) return true;
14
15     // WARUNEK 1: ktorys z koncow odc. nalezy do drugiego odc.
16     // Czy P1 lezy na P3P4 || Czy P2 lezy na P3P4 || Czy P3 lezy na P1P2 || Czy P4
17     // lezy na P1P2
18     if(is_point_on_segment(p3, p4, p1) || is_point_on_segment(p3, p4, p2) ||
19        is_point_on_segment(p1, p2, p3) || is_point_on_segment(p1, p2, p4)) {
20         return true;
21     }
22     return false;
23 }
```

Plik źródłowy 5: /Struktury/Funkcje/do_segments_intersect.cpp

2.5 distance()

Funkcja oblicza kwadrat odległości punktu a od punktu b według metryki euklidesowej

```
1 #include "distance.h"
2
3 double distance(Point a, Point b) {
4     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
5 }
```

Plik źródłowy 6: /Struktury/Funkcje/distance.cpp

2.6 is_point_in_polygon()

Funkcja implementuje algorytm z wykładu sprawdzający czy punkt p należy do listy wierzchołków (punktów *Point*) wielokąta *polygon*

Ustalmy $u = \min(x : x \in \text{polygon})$, $v = \max(x : x \in \text{polygon})$

Jeżeli $p.x < u$ lub $p.x > v$ to p nie leży w wielokącie *polygon*

Jeżeli p leży na dowolnej krawędzi wielokąta *polygon* to p leży w wielokącie *polygon*

Niech $q = (v + 1, p.y)$ i $l = 0$, i dla każdej krawędzi $p.y$ należy do domknięto-otwartego przedziału współrzędnych y tej krawędzi oraz pq ją przecina, to zwiększ l o

Jeżeli l jest nieparzyste, to punkt p leży w wielokącie *polygon*

```
1 #include "is_point_in_polygon.h"
2
3 bool is_point_in_polygon(Point p, const std::vector<Point>& polygon){
4     double u = polygon[0].x;
5     double v = polygon[0].x;
6     int n = polygon.size();
7     for(int i = 0; i < n; i++){
```

```

8      u = std::min(u, polygon[i].x);
9      v = std::max(v, polygon[i].x);
10  }
11
12  if((p.x < u) || (p.x > v)) return false;
13
14  int l = 0;
15
16  Point q = Point(v + 1, p.y);
17
18  for(int i = 0; i < n-1; i++){
19      if(is_point_on_segment(polygon[i], polygon[i+1], p)) return true;
20      if((p.y >= std::min(polygon[i].y, polygon[i+1].y) && (p.y < std::max(polygon
21  [i].y, polygon[i+1].y)))){
22          if(do_segments_intersect(polygon[i], polygon[i+1], p, q)) l++;
23      }
24  }
25
26  if(l % 2 == 1) return true;
27  return false;
}

```

Plik źródłowy 7: /Struktury/Funkcje/is_point_in_polygon.cpp

2.7 graham_scan()

Funkcja implementuje algorytm Grahama do wyznaczenia otoczki wypukłej

Pierw sortujemy rosnąco punkty względem $p.y$, gdy $p.y$ taki sam to względem $p.x$

Następnie usuwamy duplikaty (te same punkty) ze zbioru

Jeżeli zbiór zawiera mniej niż 4 unikalne elementy to zwracamy ten zbiór

Następnie sortujemy zbiór rosnąco względem współrzędnych kątowych, kolejno usuwamy bliższe punkty z tą samą współrzędną kątową co dalszy punkt

Tworzymy stos i dodajemy do niego pierwsze trzy punkty oraz dla każdego kolejnego punktu p_i z wcześniej przygotowanej tablicy dopóki następnik i następnik następnika punktu p_i nie jest skretem w lewo to usuwamy element ze stosu, następnie dodajemy p_i

Ostatecznym wynikiem jest zbiór punktów otoczki wypukłej zbioru wejściowego

```

1  #include "graham_scan.h"
2
3  // https://en.wikipedia.org/wiki/Graham_scan
4  std::vector<Point> graham_scan(std::vector<Point> points){
5      /// KROK 1: USUNIECIE DUPLIKATOW I USTAWIENIE NAJBЛИЗШЕГО
6      /// PUNKTU JAKO PIERWSZY (NAJPIERW NA Y, W RAZIE REMISU NA X)
7
8      std::sort(points.begin(), points.end(), [](const Point& a, const Point& b) {
9          int cmpY = cmpDouble(a.y, b.y);
10         if (cmpY != 0) return cmpY < 0;
11         return cmpDouble(a.x, b.x) < 0;
12     });
13
14     points.erase(std::unique(points.begin(), points.end(), [](const Point& a, const
15     Point& b) {
16         return cmpDouble(a.x, b.x) == 0 && cmpDouble(a.y, b.y) == 0;
17     }), points.end());
18
19     /// PRZYPADKI, GDY JEST PONIZEJ 3 PUNKTOW
20     if (points.size() < 3) {
21         if (points.size() == 0) return {};
22         if (points.size() == 1 || points.size() == 2) return points;
23     }
24
25     /// KROK 2: SORTUJEMY I DOSTOSOWUJEMY LISTE PUNKTOW ZE WZGLEDU NA WSPOLRZEDNE
26     KATOWE
27
28     /// NAJPIERW SORTUJEMY LISTE PUNKTOW

```

```

27     std::sort(points.begin() + 1, points.end(), [&](const Point& pi, const Point& pj)
28     {
29         double d = det(points[0], pi, pj);
30         if (sgn(d) == 0) {
31             int cmpDist = cmpDouble(distance(points[0], pi), distance(points[0], pj))
32             ;
33             if (cmpDist != 0) return cmpDist < 0;
34             int cmpX = cmpDouble(pi.x, pj.x);
35             if (cmpX != 0) return cmpX < 0;
36             return cmpDouble(pi.y, pj.y) < 0;
37         }
38         return sgn(d) > 0;
39     });
40
41     /// USUWAMY Z LISTY BLIZSZE PUNKTY Z TA SAMA WSPOLRZEDNA KATOWA CO DALSZY PUNKT
42     std::vector<Point> furtherPoints;
43     furtherPoints.push_back(points[0]);
44
45     for(unsigned int i = 1; i < points.size(); i++){
46         while(i + 1 < points.size() && sgn(det(points[0], points[i], points[i+1])) ==
47             0)
48             i++;
49         furtherPoints.push_back(points[i]);
50     }
51
52     points = furtherPoints;
53
54     /// KROK 3: Utworzenie stosu i dodanie 3 pierwszych punktów
55     if (points.size() < 3) return points;
56
57     std::vector<Point> stack;
58     stack.push_back(points[0]);
59     stack.push_back(points[1]);
60     stack.push_back(points[2]);
61
62     /// KROK 4: Ostateczne dobranie punktów otoczki
63     for (unsigned int i = 3; i < points.size(); i++) {
64         while (stack.size() >= 2 && sgn(det(stack[stack.size() - 2], stack.back(),
65             points[i])) <= 0) {
66             stack.pop_back();
67         }
68         stack.push_back(points[i]);
69     }
70     return stack;
71 }

```

Plik źródłowy 8: /Struktury/Funkcje/graham_scan.cpp

2.8 KMP()

-> Opis

```

1  #include "KMP.h"
2
3  std::vector<int> buildPi(std::string P){
4      std::vector<int> pi;
5      //unsigned int m = P.size();
6      for(unsigned int i{}; i<P.size(); i++){
7          pi.push_back(0);
8      }
9
10     unsigned int k = 0;
11     for(unsigned int q = 1; q<P.size(); q++){
12         while((k > 0) && P[k] != P[q]){

```

```

13         k = pi[k-1];
14     }
15     if(P[k] == P[q]){
16         k++;
17     }
18     pi[q] = k;
19 }
20
21 return pi;
22 }
23
24 std::vector<int> KMP(std::string T, std::string P){
25     std::vector<int> pi = buildPi(P);
26     unsigned int q = 0;
27     unsigned int n = T.size();
28     unsigned int m = P.size();
29     std::vector<int> shifts = {};
30
31     for(unsigned int i{}; i<n; i++){
32         while((q > 0) && (P[q] != T[i])){
33             q = pi[q-1];
34         }
35         if(P[q] == T[i]){
36             q++;
37         }
38         if(q == m){
39             shifts.push_back(i - m + 1);
40             q = pi[q-1];
41         }
42     }
43     return shifts;
44 }

```

Plik źródłowy 9: /Struktury/Funkcje/KMP.cpp

3 Struktury

3.1 Point

Struktura naszego punktu w układzie kartezjańskim, zawiera ona definicję oraz operatory do porównywania dwóch punktów oraz wykonywania działań na nich

Operator mniejszości: " $p_1 < p_2$ " sprawdza czy punkt p_1 na lewo od p_2 . Jeżeli mają tę samą współrzędną $p_1.x$, to sprawdza czy p_1 jest poniżej p_2

Operator większości: $p_1 > p_2$ sprawdza $p_2 < p_1$

Operator równości: $p_1 == p_2$ sprawdza czy współrzędne x i y są takie same

Operator nierówności: $p_1 != p_2$ [$\neq$] zwraca przeciwieństwo $p_1 == p_2$

Operator mniejszy-równy: $p_1 <= p_2$ [$\leq$] zwraca $p_1 < p_2 \vee p_1 == p_2$

Operator większy-równy: $p_1 >= p_2$ [$\geq$] zwraca $p_1 > p_2 \vee p_1 == p_2$

Operator przypisania: $p_1 = p_2$ przypisuje punktowi p_1 odpowiednio $p_1.x = p_2.x$ oraz $p_1.y = p_2.y$

Wypisanie punktu: $<< p$ wypisuje punkt p w postaci $(p_1.x, p_1.y)$

```

1 #include "Point.h"
2
3 Point::Point(double x, double y) : x(x), y(y) {}
4
5 // Operator mniejszosci
6 bool Point::operator<(const Point& other) const {
7     int cmpX = cmpDouble(x, other.x);
8     if (cmpX != 0) {
9         return cmpX < 0;
10    }
11    return cmpDouble(y, other.y) < 0;
12 }
13

```

```

14 // Operator wielkosci
15 bool Point::operator>(const Point& other) const {
16     return other < *this;
17 }
18
19 // Operator rownosci
20 bool Point::operator==(const Point& other) const {
21     return cmpDouble(x, other.x) == 0 && cmpDouble(y, other.y) == 0;
22 }
23
24 // Operator nierownosci
25 bool Point::operator!=(const Point& other) const {
26     return !(*this == other);
27 }
28
29 // Operator mniejszosci lub rownosci
30 bool Point::operator<=(const Point& other) const {
31     return (*this < other) || (*this == other);
32 }
33
34 // Operator wielkosci lub rownosci
35 bool Point::operator>=(const Point& other) const {
36     return (*this > other) || (*this == other);
37 }
38
39 // Operator przypisania
40 Point& Point::operator=(const Point& other) {
41     if(this != &other){
42         x = other.x;
43         y = other.y;
44     }
45
46     return *this;
47 }
48
49 // Wypisanie punktu
50 std::ostream& operator<<(std::ostream& stream, const Point& p){
51     stream << "(" << p.x << ", " << p.y << ")";
52     return stream;
53 }

```

Plik źródłowy 10: /Struktury/Point.cpp

3.2 Huffman.cpp

Bardziej niżeli struktura to zbiór oddzielnych funkcji przy użyciu których korzystamy z algorytmu Huffmana do kodowania tekstu (Algorytm Huffmana)

Poniżej zawartość pliku nagłówkowego dla łatwiejszego zrozumienia kodu źródłowego

```

1 #ifndef HUFFMAN_H
2 #define HUFFMAN_H
3
4 #include <memory>
5 #include <queue>
6 #include <vector>
7 #include <string>
8 #include <iostream>
9 #include <algorithm>
10
11 struct HuffmanNode{
12     char c;
13     int freq;
14     bool isLeaf;
15
16     std::unique_ptr<HuffmanNode> left;
17     std::unique_ptr<HuffmanNode> right;
18

```

```

19 HuffmanNode() : c(0), freq(0), isLeaf(false), left(nullptr), right(nullptr) {}
20 HuffmanNode(char character, int frequency) : c(character), freq(frequency),
    isLeaf(true), left(nullptr), right(nullptr) {}
21 };
22
23 struct HuffmanCode{
24     char c;
25     std::string code;
26
27     HuffmanCode(char character, const std::string& huffmanCode) : c(character), code(
        huffmanCode) {}
28 };
29
30 std::unique_ptr<HuffmanNode> buildHuffmanTree(const std::vector<char>& C, const std::
    vector<int>& F);
31 std::vector<HuffmanCode> generateHuffmanCodes(const std::unique_ptr<HuffmanNode>&
    node, const std::string& prefix = "");
32 std::vector<HuffmanCode> Huffman(const std::vector<char>& C, const std::vector<int>&
    F);
33 void HuffmanPrint(const std::vector<HuffmanCode>& codes);
34 bool HuffmanEquals(const std::vector<HuffmanCode>& a, const std::vector<HuffmanCode>&
    b);
35 std::vector<HuffmanCode> HuffmanSort(const std::vector<HuffmanCode>& codes);
36
37 #endif

```

Plik źródłowy 11: /Struktury/Huffman.h

3.2.1 HuffmanNode

Struktura która przechowuje znak *c*, liczbę wystąpień *freq*, flagę czy jest liściem *isLeaf*, wskaźnik do lewego dziecka *left* oraz do prawego *right*

Dodatkowo konstruktory do sprawnego tworzenia obiektów

3.2.2 HuffmanCode

Struktura która zawiera znak *c* oraz kod jego odpowiedni kod Huffmana *code*

Dodatkowo konstruktory do sprawnego tworzenia obiektów

3.2.3 buildHuffmanTree()

Funkcja, która jak nazwa wskazuje, tworzy drzewo kodowań Huffmana

Definiujemy funkcję porównywania struktur *HuffmanNode* która sprawdza która jest większa " > " według pola *HuffmanNode.freq*

Następnie sprawdzamy czy któraś z tablic wejściowych (tablica znaków lub tablica liczby wystąpień) jest pusta, jak tak to zwracamy pusty wskaźnik

Tworzymy kolejkę priorytetową typu min z priorytetem *HuffmanNode.freq*

Tworzymy struktury *HuffmanNode* na podstawie danych wejściowych i dodajemy ich wskaźniki do kolejki priorytetowej

Dopóki rozmiar kolejki priorytetowej jest większy od 1 to wybieramy dwie struktury o najniższej wartości *freq* (najwyższy priorytet) jako odpowiednio *left* i *right* oraz tworzymy nowy węzeł z wskaźnikami do *left* i *right* i odkładamy go do kolejki

Po wyjściu z pętli jeżeli kolejka jest pusta to zwracamy pusty wskaźnik, a w przeciwnym wypadku zwracamy jedyny element kolejki który jest korzeniem drzewa Huffmana

3.2.4 generateHuffmanCodes()

Funkcja która rekurencyjnie na podstawie struktury *HuffmanNode* i podanego prefixu tworzy kodowanie Huffmana

Tworzymy tablicę *HuffmanCode* i następnie jeżeli dany element jest liściem to dodajemy znak

c i jego kod *prefix* do tablicy kodów, jeżeli nie jest liściem, to dla lewego dziecka wykonujemy *generateHuffmanCodes* na tym dziecku z tym samym *prefixem* plus 0 i zwróconą wartość dodajemy do tablicy kodów, a dla prawego dziecka tak samo tylko wywołujemy funkcję *generateHuffmanCodes* od prawego dziecka i z *prefixem* plus 1 oraz dodajemy do tablicy kodów. Ostatecznie zwracamy tablicę kodów.

3.2.5 Huffman()

Wywołuje funkcję *buildHuffmanTree* i na jej wyniku wywołuje funkcję *generateHuffmanCodes* i zwraca jej wynik.

3.2.6 HuffmanPrint()

Wypisuje *HuffmanCode* w postaci *znak : kod*.

3.2.7 HuffmanEquals()

Funkcja sprawdza czy dwa kody Huffmana są równe w sensie mają ten sam znak c i ten sam kod Huffmana.

3.2.8 HuffmanSort()

Funkcja sortuje tablicę kodów Huffmana alfabetycznie według znaku c .

3.2.9 Kod źródłowy

```

1 #include "Huffman.h"
2
3 std::unique_ptr<HuffmanNode> buildHuffmanTree(const std::vector<char>& C, const std::
  vector<int>& F){
4     auto cmp = [](const std::unique_ptr<HuffmanNode>& a, const std::unique_ptr<
  HuffmanNode>& b){
5         return a->freq > b->freq;
6     };
7
8     if((C.size() != F.size()) || C.empty() || F.empty()){
9         return nullptr;
10    }
11
12    std::priority_queue<std::unique_ptr<HuffmanNode>, std::vector<std::unique_ptr<
  HuffmanNode>>, decltype(cmp)> pq(cmp);
13
14    for(size_t i = 0; i < C.size(); i++){
15        pq.push(std::move(std::make_unique<HuffmanNode>(C[i], F[i])));
16    }
17
18    while(pq.size() > 1){
19        auto left = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
20        pq.pop();
21        auto right = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));
22        pq.pop();
23
24        auto parent = std::make_unique<HuffmanNode>();
25        parent->freq = left->freq + right->freq;
26        parent->isLeaf = false;
27        parent->left = std::move(left);
28        parent->right = std::move(right);
29
30        pq.push(std::move(parent));
31    }
32
33    if(pq.empty()) return nullptr;
34    auto root = std::move(const_cast<std::unique_ptr<HuffmanNode>&>(pq.top()));

```

```

35     pq.pop();
36
37     return root;
38 }
39
40 std::vector<HuffmanCode> generateHuffmanCodes(const std::unique_ptr<HuffmanNode>&
41 node, const std::string& prefix){
42     std::vector<HuffmanCode> codes;
43
44     if(node->isLeaf){
45         codes.emplace_back(node->c, prefix);
46     } else{
47         if(node->left) {
48             auto leftCodes = generateHuffmanCodes(node->left, prefix + "0");
49             codes.insert(codes.end(), leftCodes.begin(), leftCodes.end());
50         }
51         if(node->right) {
52             auto rightCodes = generateHuffmanCodes(node->right, prefix + "1");
53             codes.insert(codes.end(), rightCodes.begin(), rightCodes.end());
54         }
55     }
56
57     return codes;
58 }
59
60 std::vector<HuffmanCode> Huffman(const std::vector<char>& C, const std::vector<int>&
61 F){
62     auto root = buildHuffmanTree(C, F);
63
64     if(root == nullptr){
65         return {};
66     }
67
68     std::vector<HuffmanCode> huffmanCodes;
69     huffmanCodes = generateHuffmanCodes(root);
70
71     return huffmanCodes;
72 }
73
74 void HuffmanPrint(const std::vector<HuffmanCode>& codes){
75     for(const auto& code : codes){
76         std::cout<<code.c<<" ";
77     }
78 }
79
80 bool HuffmanEquals(const std::vector<HuffmanCode>& a, const std::vector<HuffmanCode>&
81 b){
82     if(a.size() != b.size()) return false;
83
84     for(size_t i = 0; i < a.size(); i++){
85         if((a[i].c != b[i].c) || (a[i].code != b[i].code)){
86             return false;
87         }
88     }
89
90     return true;
91 }
92
93 std::vector<HuffmanCode> HuffmanSort(const std::vector<HuffmanCode>& codes){
94     if(codes.empty()) return {};
95
96     std::vector<HuffmanCode> sortedCodes = codes;
97     std::sort(sortedCodes.begin(), sortedCodes.end(), [](const HuffmanCode& a, const
98 HuffmanCode& b){
99         return a.c < b.c;
100     });
101     return sortedCodes;

```

98 }

Plik źródłowy 12: /Struktury/Huffman.cpp

3.3 Edge.h

-> Opis

```
1 #ifndef EDGE_H
2 #define EDGE_H
3
4 struct Edge {
5     int to;           // dokad idzie
6     int flow;         // przeplyw
7     int capacity;     // pojemnosc
8     int rev;          // ile wezlow ma dany node
9     int cost;         // ile posilkow musi zjesc krasnolud, zeby przebyc droge
10
11     Edge(int to, int capacity, int rev, int cost) :
12         to(to), flow(0), capacity(capacity), rev(rev), cost(cost) {}
13 };
14
15 #endif
```

Plik źródłowy 13: /Struktury/Edge.h

3.4 Graph.cpp

->Opis

```
1 #include "Graph.h"
2
3 void Graph::add_edge(int from, int to, int capacity, int cost) {
4     graph[from].push_back(Edge(to, capacity, (int)graph[to].size(), cost));
5     graph[to].push_back(Edge(from, 0, (int)graph[from].size() - 1, -cost));
6 }
7
8 void Graph::clear() {
9     for (auto& edge : graph) edge.clear();
10 }
```

Plik źródłowy 14: /Struktury/Graph.cpp

3.5 Kopalnia.cpp

-> Opis

```
1 #include "Kopalnia.h"
2
3 Kopalnia::Kopalnia()
4     : id(0), location(0.0, 0.0), resourceType(""), capacity(0), assignedDwarves({})
5     {}
6
7 Kopalnia::Kopalnia(int id, const Point& location, const std::string& resourceType,
8     int capacity, const std::vector<int>& assignedDwarves)
9     : id(id),
10     location(location),
11     resourceType(resourceType),
12     capacity(capacity),
13     assignedDwarves(assignedDwarves) {}
14
15 int Kopalnia::getId() const {
16     return id;
17 }
18
19 const Point& Kopalnia::getLocation() const {
20     return location;
21 }
```

```

20 }
21
22 double Kopalnia::getX() const {
23     return location.x;
24 }
25
26 double Kopalnia::getY() const {
27     return location.y;
28 }
29
30 const std::string& Kopalnia::getResourceType() const {
31     return resourceType;
32 }
33
34 int Kopalnia::getCapacity() const {
35     return capacity;
36 }
37
38 const std::vector<int>& Kopalnia::getAssignedDwarves() const {
39     return assignedDwarves;
40 }
41
42 void Kopalnia::setId(int newId) {
43     this->id = newId;
44 }
45
46 void Kopalnia::setLocation(const Point& newLocation) {
47     this->location = newLocation;
48 }
49
50 void Kopalnia::setX(double newX) {
51     this->location.x = newX;
52 }
53
54 void Kopalnia::setY(double newY) {
55     this->location.y = newY;
56 }
57
58 void Kopalnia::setResourceType(const std::string& newResourceType) {
59     this->resourceType = newResourceType;
60 }
61
62 void Kopalnia::setCapacity(int newCapacity) {
63     this->capacity = newCapacity;
64 }
65
66 void Kopalnia::setAssignedDwarves(const std::vector<int>& newAssignedDwarves) {
67     this->assignedDwarves = newAssignedDwarves;
68 }
69
70 int Kopalnia::getAvailableCapacity() const {
71     return capacity - static_cast<int>(assignedDwarves.size());
72 }
73
74 bool Kopalnia::hasAvailableSpace() const {
75     return getAvailableCapacity() > 0;
76 }
77
78 bool Kopalnia::isInUse() const {
79     return !assignedDwarves.empty();
80 }
81
82 void Kopalnia::addDwarf(int dwarfId) {
83     if (!hasAvailableSpace()) {
84         return;
85     }
86

```

```

87     if (std::find(assignedDwarves.begin(), assignedDwarves.end(), dwarfId) !=
88         assignedDwarves.end()) {
89         return;
90     }
91     assignedDwarves.push_back(dwarfId);
92 }
93
94 void Kopalnia::removeDwarf(int dwarfId) {
95     auto it = std::find(assignedDwarves.begin(), assignedDwarves.end(), dwarfId);
96
97     if (it != assignedDwarves.end()) {
98         assignedDwarves.erase(it);
99     }
100 }

```

Plik źródłowy 15: /Struktury/Kopalnia.cpp

3.6 Krasnal.cpp

-> Opis

```

1 #include "Krasnal.h"
2
3 Krasnal::Krasnal()
4     : id(0), home(0.0, 0.0), skills({}), preferredResource(""),
5       assignedToPreferredResource(false) {}
6
7 Krasnal::Krasnal(int id, const Point& home,
8                 const std::vector<std::string>& skills,
9                 const std::string& preferredResource,
10                bool assignedToPreferredResource)
11     : id(id),
12       home(home),
13       skills(skills),
14       preferredResource(preferredResource),
15       assignedToPreferredResource(assignedToPreferredResource) {}
16
17 int Krasnal::getId() const{
18     return id;
19 }
20
21 const Point& Krasnal::getHome() const{
22     return home;
23 }
24
25 double Krasnal::getHomeY() const{
26     return home.y;
27 }
28
29 double Krasnal::getHomeX() const{
30     return home.x;
31 }
32
33 const std::vector<std::string>& Krasnal::getSkills() const{
34     return skills;
35 }
36
37 const std::string& Krasnal::getPreferredResource() const{
38     return preferredResource;
39 }
40
41 bool Krasnal::isAssignedToPreferredResource() const{
42     return assignedToPreferredResource;
43 }
44
45 void Krasnal::setId(int newId){

```

```

45     this->id = newId;
46 }
47
48 void Krasnal::setHome(const Point& newHome){
49     this->home = newHome;
50 }
51
52 void Krasnal::setHomeX(double newX){
53     this->home.x = newX;
54 }
55
56 void Krasnal::setHomeY(double newY){
57     this->home.y = newY;
58 }
59
60 void Krasnal::setSkills(const std::vector<std::string>& newSkills){
61     this->skills = newSkills;
62 }
63 void Krasnal::setPreferredResource(const std::string& newPreferredResource){
64     this->preferredResource = newPreferredResource;
65 }
66 void Krasnal::setAssignedToPreferredResource(bool assignedToPreferredResource){
67     this->assignedToPreferredResource = assignedToPreferredResource;
68 }

```

Plik źródłowy 16: /Struktury/Krasnal.cpp

3.7 MinCostMaxFlow.cpp

-> Opis

```

1 #include "MinCostMaxFlow.h"
2 #include <algorithm>
3 #include <limits>
4
5 using namespace std;
6
7 static bool bellman_ford(const Graph& graph, int source, int sink,
8     vector<long long>& dist, vector<int>& parentVertex, vector<int>& parentEdge) {
9
10     const long long INF = numeric_limits<long long>::max() / 4;
11
12     dist.assign(graph.n, INF);
13     //Wierzecholek
14     parentVertex.assign(graph.n, -1);
15     parentEdge.assign(graph.n, -1);
16     dist[source] = 0;
17
18     for ( int i = 0; i < graph.n - 1; i++ ) {
19
20         bool changed = false;
21
22         for ( int j = 0; j < graph.n; j++ ) {
23             if ( dist[j] == INF ) {
24                 continue;
25             }
26
27             for (size_t edgeIndex = 0; edgeIndex < graph.graph[j].size(); edgeIndex++) {
28                 const Edge& edge = graph.graph[j][edgeIndex];
29
30                 int residual = edge.capacity - edge.flow;
31                 if (residual <= 0) {
32                     continue;
33                 }
34
35                 long long candidate = dist[j] + edge.cost;
36

```

```

37         if (candidate < dist[edge.to]) {
38             dist[edge.to] = candidate;
39             parentVertex[edge.to] = j;
40             parentEdge[edge.to] = static_cast<int>(edgeIndex);
41             changed = true;
42         }
43     }
44 }
45 if (!changed) {
46     break;
47 }
48
49
50 }
51
52 return dist[sink] != INF;
53
54 }
55
56 MinCostMaxFlowResult min_cost_max_flow(Graph& graph, int source, int sink, int
maxFlow) {
57     MinCostMaxFlowResult result;
58
59     if ( source < 0 || source >= graph.n || sink < 0 || sink >= graph.n ) {
60         return result;
61     }
62
63     if ( source == sink || maxFlow <= 0 ) {
64         return result;
65     }
66
67     vector<long long> dist;
68     vector<int> parentVertex;
69     vector<int> parentEdge;
70
71     while (result.flow < maxFlow && bellman_ford(graph, source, sink, dist,
parentVertex, parentEdge)) {
72         int pushedFlow = maxFlow - result.flow;
73
74         for ( int vertex = sink; vertex != source; vertex = parentVertex[vertex] ) {
75             int previousVertex = parentVertex[vertex];
76             int edgeIndex = parentEdge[vertex];
77             const Edge& edge = graph.graph[previousVertex][edgeIndex];
78
79             pushedFlow = min(pushedFlow, edge.capacity - edge.flow);
80         }
81
82         if (pushedFlow <= 0) {
83             break;
84         }
85         for ( int vertex = sink; vertex != source; vertex = parentVertex[vertex] ) {
86             int previousVertex = parentVertex[vertex];
87             int edgeIndex = parentEdge[vertex];
88             Edge& edge = graph.graph[previousVertex][edgeIndex];
89             Edge& reverseEdge = graph.graph[vertex][edge.rev];
90             edge.flow += pushedFlow;
91             reverseEdge.flow -= pushedFlow;
92         }
93         result.flow += pushedFlow;
94         result.cost += static_cast<long long>(pushedFlow) * dist[sink];
95     }
96     return result;
97 }
98 }

```

Plik źródłowy 17: /Struktury/MinCostMaxFlow.cpp

3.8 SegmentTree.cpp

Struktura danych jest drzewem przedziałowym, służącym do efektywnego wykonywania operacji wyszukiwania maksimum na zadanym przedziale tablicy oraz aktualizacji pojedynczych elementów.

3.8.1 build()

Proces budowy drzewa realizowany jest rekurencyjnie. Dla przedziału zawierającego jeden element wartość węzła ustawiana jest bezpośrednio na wartość elementu tablicy. W.p.p. przedział dzielony jest na dwie części. Następnie rekurencyjnie budowane są poddrzewa reprezentujące oba fragmenty.

3.8.2 query()

Umożliwia znalezienie największej wartości w zadanym przedziale indeksów. Algorytm analizuje relację pomiędzy aktualnym przedziałem reprezentowanym przez węzeł a zakresem zapytania. W przypadku braku części wspólnej zwracana jest najmniejsza możliwa wartość typu całkowitego. Jeżeli aktualny przedział w całości zawiera się w zakresie zapytania, zwracana jest wartość zapisana w danym węźle. W przeciwnym przypadku wykonywane są rekurencyjne zapytania dla lewego i prawego poddrzewa, a wyniki łączone są za pomocą operacji maksimum.

3.8.3 update()

Pozwala zmienić wartość pojedynczego elementu tablicy. Algorytm przechodzi od korzenia drzewa do odpowiedniego liścia reprezentującego aktualizowany indeks. Po zmianie wartości liścia następuje ponowne przeliczenie wartości wszystkich odwiedzonych wcześniej węzłów, dzięki czemu drzewo zachowuje poprawność przechowywanych danych

```
1 #include "SegmentTree.h"
2 #include <iostream>
3 using namespace std;
4
5 // !!! Indeksowanie od 0
6
7 void SegmentTree::build(int v, int l, int r)
8 {
9     if (l == r)
10     {
11         tree[v] = A[l];
12         return;
13     }
14
15     int mid = (l + r) / 2;
16     build(2 * v + 1, l, mid);
17     build(2 * v + 2, mid + 1, r);
18
19     tree[v] = max(tree[2 * v + 1], tree[2 * v + 2]);
20 }
21
22 int SegmentTree::query(int v, int l, int r, int ql, int qr)
23 {
24     if (r < ql || qr < l)
25         return numeric_limits<int>::lowest();
26     ; // rozłączne
27     if (ql <= l && r <= qr)
28         return tree[v]; // zawarty
29
30     int mid = (l + r) / 2;
31     int x = query(2 * v + 1, l, mid, ql, qr);
32     int y = query(2 * v + 2, mid + 1, r, ql, qr);
33
34     return max(x, y);
35 }
36
37 void SegmentTree::update(int v, int l, int r, int idx, int x)
```

```

38 {
39     if (l == r)
40     {
41         tree[v] = x;
42         return;
43     }
44
45     int mid = (l + r) / 2;
46     if (idx <= mid)
47         update(2 * v + 1, l, mid, idx, x);
48     else
49         update(2 * v + 2, mid + 1, r, idx, x);
50
51     tree[v] = max(tree[2 * v + 1], tree[2 * v + 2]);
52 }
53
54 SegmentTree::SegmentTree(const vector<int> &a) : A(a), n(a.size())
55 {
56     tree.resize(4 * n);
57     build(0, 0, n - 1);
58 }
59
60 void SegmentTree::update(int i, int val)
61 {
62     update(0, 0, n - 1, i, val);
63 }
64
65 int SegmentTree::query(int l, int r)
66 {
67     return query(0, 0, n - 1, l, r);
68 }

```

Plik źródłowy 18: /Struktury/SegmentTree.cpp

3.9 Straznik.cpp

-> Opis

```

1 #include "Straznik.h"
2
3 Straznik::Straznik() : id(0), loudness(0) {}
4
5 Straznik::Straznik(int id, int loudness) : id(id), loudness(loudness) {}
6
7 int Straznik::getId() const {
8     return id;
9 }
10
11 int Straznik::getLoudness() const {
12     return loudness;
13 }
14
15 void Straznik::setLoudness(int newLoudness) {
16     this->loudness = newLoudness;
17 }
18
19 void Straznik::setId(int newId) {
20     this->id = newId;
21 }

```

Plik źródłowy 19: /Struktury/Straznik.cpp

4 Solvery

4.1 BorderPatrolSolver

Klasa *BorderPatrolSolver* spina problem wyznaczania trasy patrolu po aktywnych kopalniach. Na wejściu otrzymuje listę punktów reprezentujących położenia kopalni, w których faktycznie pracują krasnale. Następnie za pomocą funkcji *graham_scan()* wyznacza otoczkę wypukłą tych punktów. Długość patrolu jest liczona jako suma długości kolejnych krawędzi tej otoczki. Funkcja *distance()* zwraca kwadrat odległości, dlatego przy sumowaniu używany jest pierwiastek.

4.1.1 calculateConvexHull()

Metoda wyznacza i zapamiętuje otoczkę wypukłą aktywnych kopalni. Jeżeli wynik został już wcześniej policzony, zwracana jest zapamiętana otoczka.

4.1.2 calculatePatrolDistance()

Metoda zwraca długość zamkniętej trasy patrolu po punktach otoczki. Dla zera lub jednego punktu zwracana jest wartość 0, ponieważ nie istnieje rzeczywista trasa do okrążenia.

4.1.3 Kod źródłowy

```
1 #ifndef BORDER_PATROL_H
2 #define BORDER_PATROL_H
3
4 #include <vector>
5 #include "Point.h"
6 #include "Funkcje/graham_scan.h"
7 #include "Funkcje/distance.h"
8 #include <cmath>
9
10 class BorderPatrolSolver {
11 private:
12     std::vector<Point> activeMines;
13     std::vector<Point> hull;
14     bool calculated;
15
16 public:
17     BorderPatrolSolver(const std::vector<Point> &mines);
18
19     std::vector<Point> calculateConvexHull();
20     double calculatePatrolDistance();
21 };
22
23
24 #endif
```

Plik źródłowy 20: /Struktury/BorderPatrolSolver.h

```
1 #include "BorderPatrolSolver.h"
2
3 BorderPatrolSolver::BorderPatrolSolver(const std::vector<Point> &mines)
4     : activeMines(mines), calculated(false) {}
5
6 std::vector<Point> BorderPatrolSolver::calculateConvexHull() {
7     if(!calculated){
8         hull = graham_scan(activeMines);
9         calculated = true;
10    }
11    return hull;
12 }
13
14 double BorderPatrolSolver::calculatePatrolDistance(){
15     if(!calculated){
```

```

16     hull = graham_scan(activeMines);
17     calculated = true;
18 }
19
20 if(hull.size() < 2) return 0.0;
21
22 double dist = 0.0;
23
24 for(size_t i = 0; i < hull.size(); i++)
25 {
26     Point a = hull[i];
27     Point b = hull[(i + 1) % hull.size()];
28     dist += std::sqrt(distance(a,b)); // bo distance liczy kwadrat odleglosci
29 }
30 return dist;
31 }

```

Plik źródłowy 21: /Struktury/BorderPatrolSolver.cpp

4.2 WorkAssignmentSolver

Klasa *WorkAssignmentSolver* rozwiązuje problem przydziału krasnali do kopalni. Problem został zamodelowany jako sieć przepływu o minimalnym koszcie. Wierzchołki grafu odpowiadają źródłu, krasnalom, kopalniom oraz ujściu. Krawędź ze źródła do krasnala ma przepustowość 1, ponieważ jeden krasnal może dostać najwyżej jedną pracę. Krawędź z kopalni do ujścia ma przepustowość równą pojemności kopalni. Krawędź z krasnala do kopalni istnieje tylko wtedy, gdy krasnal posiada kompetencję wymaganą przez surowiec kopalni.

4.2.1 Model kosztu

Koszt krawędzi krasnal–kopalnia składa się z odległości domu krasnala od kopalni oraz ewentualnej kary za brak preferowanego surowca. Odległość jest liczona jako odległość euklidesowa i skalowana do liczby całkowitej, ponieważ *MinCostMaxFlow* operuje na kosztach typu *int*. Kara za brak preferencji jest większa niż maksymalna możliwa różnica sumy odległości, dzięki czemu solver najpierw preferuje pracę przy ulubionym surowcu, a dopiero potem minimalizuje dystans.

4.2.2 Wynik

Wynikiem działania solvera jest struktura *WorkAssignmentResult*, która zawiera listę przydziałów, listę nieprzydzielonych krasnali, liczbę przydziałów, liczbę przydziałów zgodnych z preferencjami oraz łączny dystans. Solver przechowuje też kopie kopalni i krasnali po aktualizacji, dzięki czemu kolejne moduły mogą korzystać z informacji o aktywnych kopalniach.

4.2.3 Kod źródłowy

```

1 #ifndef WORKASSIGNMENTSOLVER_H
2 #define WORKASSIGNMENTSOLVER_H
3
4 #include "../Kopalnia.h"
5 #include "../Krasnal.h"
6 #include "../MinCostMaxFlow.h"
7
8 #include <string>
9 #include <vector>
10
11 struct WorkAssignment {
12     int dwarfId;
13     int mineId;
14     std::string resourceType;
15     double distance;
16     bool preferredResource;
17
18     WorkAssignment(int dwarfId, int mineId, const std::string& resourceType,

```

```

19         double distance, bool preferredResource);
20     };
21
22     struct WorkAssignmentResult {
23         std::vector<WorkAssignment> assignments;
24         std::vector<int> unassignedDwarfIds;
25         int assignedCount;
26         int preferredAssignedCount;
27         double totalDistance;
28         long long optimizationCost;
29
30         WorkAssignmentResult();
31         bool allDwarvesAssigned() const;
32     };
33
34     class WorkAssignmentSolver {
35     private:
36         std::vector<Krasnal> dwarves;
37         std::vector<Kopalnia> mines;
38
39         int sourceNode() const;
40         int sinkNode() const;
41         int dwarfNode(size_t dwarfIndex) const;
42         int mineNode(size_t mineIndex) const;
43
44         bool canWorkAtMine(const Krasnal& dwarf, const Kopalnia& mine) const;
45         bool isPreferredMine(const Krasnal& dwarf, const Kopalnia& mine) const;
46         double calculateTravelDistance(const Krasnal& dwarf, const Kopalnia& mine) const;
47         int calculateTravelCost(const Krasnal& dwarf, const Kopalnia& mine) const;
48         int calculatePreferencePenalty() const;
49
50         void clearAssignments();
51         void buildGraph(Graph& graph, int preferencePenalty) const;
52         void extractAssignments(const Graph& graph, WorkAssignmentResult& result);
53
54     public:
55         WorkAssignmentSolver(const std::vector<Krasnal>& dwarves,
56                             const std::vector<Kopalnia>& mines);
57
58         WorkAssignmentResult solve();
59
60         const std::vector<Krasnal>& getDwarves() const;
61         const std::vector<Kopalnia>& getMines() const;
62     };
63
64 #endif

```

Plik źródłowy 22: /Struktury/WorkAssignmentSolver/WorkAssignmentSolver.h

```

1 #include "WorkAssignmentSolver.h"
2
3 #include "../Funkcje/distance.h"
4
5 #include <algorithm>
6 #include <cmath>
7 #include <limits>
8
9 namespace {
10     const int DISTANCE_SCALE = 1000;
11 }
12
13 WorkAssignment::WorkAssignment(int dwarfId, int mineId, const std::string&
14     resourceType,
15     double distance, bool preferredResource)
16     : dwarfId(dwarfId),
17       mineId(mineId),
18       resourceType(resourceType),
19       distance(distance),

```

```

19     preferredResource(preferredResource) {}
20
21 WorkAssignmentResult::WorkAssignmentResult()
22     : assignedCount(0),
23       preferredAssignedCount(0),
24       totalDistance(0.0),
25       optimizationCost(0) {}
26
27 bool WorkAssignmentResult::allDwarvesAssigned() const {
28     return unassignedDwarfIds.empty();
29 }
30
31 WorkAssignmentSolver::WorkAssignmentSolver(const std::vector<Krasnal>& dwarves,
32                                             const std::vector<Kopalnia>& mines)
33     : dwarves(dwarves), mines(mines) {}
34
35 int WorkAssignmentSolver::sourceNode() const {
36     return 0;
37 }
38
39 int WorkAssignmentSolver::sinkNode() const {
40     return 1 + static_cast<int>(dwarves.size()) + static_cast<int>(mines.size());
41 }
42
43 int WorkAssignmentSolver::dwarfNode(size_t dwarfIndex) const {
44     return 1 + static_cast<int>(dwarfIndex);
45 }
46
47 int WorkAssignmentSolver::mineNode(size_t mineIndex) const {
48     return 1 + static_cast<int>(dwarves.size()) + static_cast<int>(mineIndex);
49 }
50
51 bool WorkAssignmentSolver::canWorkAtMine(const Krasnal& dwarf, const Kopalnia& mine)
52     const {
53     const std::vector<std::string>& skills = dwarf.getSkills();
54     return std::find(skills.begin(), skills.end(), mine.getResourceType()) != skills.
55         end();
56 }
57
58 bool WorkAssignmentSolver::isPreferredMine(const Krasnal& dwarf, const Kopalnia& mine)
59     const {
60     return dwarf.getPreferredResource() == mine.getResourceType();
61 }
62
63 double WorkAssignmentSolver::calculateTravelDistance(const Krasnal& dwarf, const
64     Kopalnia& mine) const {
65     return std::sqrt(distance(dwarf.getHome(), mine.getLocation()));
66 }
67
68 int WorkAssignmentSolver::calculateTravelCost(const Krasnal& dwarf, const Kopalnia&
69     mine) const {
70     return static_cast<int>(std::round(calculateTravelDistance(dwarf, mine) *
71     DISTANCE_SCALE));
72 }
73
74 int WorkAssignmentSolver::calculatePreferencePenalty() const {
75     int maxTravelCost = 0;
76
77     for (const Krasnal& dwarf : dwarves) {
78         for (const Kopalnia& mine : mines) {
79             if (canWorkAtMine(dwarf, mine)) {
80                 maxTravelCost = std::max(maxTravelCost, calculateTravelCost(dwarf,
81                 mine));
82             }
83         }
84     }
85
86     if (maxTravelCost == 0) {

```

```

80     return 1;
81 }
82
83 const long long penalty = static_cast<long long>(maxTravelCost) *
84     static_cast<long long>(dwarves.size()) + 1;
85
86 if (penalty > std::numeric_limits<int>::max()) {
87     return std::numeric_limits<int>::max();
88 }
89
90 return static_cast<int>(penalty);
91 }
92
93 void WorkAssignmentSolver::clearAssignments() {
94     for (Krasnal& dwarf : dwarves) {
95         dwarf.setAssignedToPreferredResource(false);
96     }
97
98     for (Kopalnia& mine : mines) {
99         mine.setAssignedDwarves(std::vector<int>{});
100     }
101 }
102
103 void WorkAssignmentSolver::buildGraph(Graph& graph, int preferencePenalty) const {
104     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
105         graph.add_edge(sourceNode(), dwarfNode(dwarfIndex), 1, 0);
106     }
107
108     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
109         for (size_t mineIndex = 0; mineIndex < mines.size(); mineIndex++) {
110             const Krasnal& dwarf = dwarves[dwarfIndex];
111             const Kopalnia& mine = mines[mineIndex];
112
113             if (!canWorkAtMine(dwarf, mine)) {
114                 continue;
115             }
116
117             long long cost = calculateTravelCost(dwarf, mine);
118             if (!isPreferredMine(dwarf, mine)) {
119                 cost += preferencePenalty;
120             }
121
122             if (cost > std::numeric_limits<int>::max()) {
123                 cost = std::numeric_limits<int>::max();
124             }
125
126             graph.add_edge(dwarfNode(dwarfIndex), mineNode(mineIndex), 1, static_cast
<int>(cost));
127         }
128     }
129
130     for (size_t mineIndex = 0; mineIndex < mines.size(); mineIndex++) {
131         graph.add_edge(mineNode(mineIndex), sinkNode(), mines[mineIndex].getCapacity
(), 0);
132     }
133 }
134
135 void WorkAssignmentSolver::extractAssignments(const Graph& graph,
WorkAssignmentResult& result) {
136     const int firstMineNode = mineNode(0);
137     const int lastMineNode = sinkNode() - 1;
138
139     std::vector<bool> assigned(dwarves.size(), false);
140
141     for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
142         const int currentDwarfNode = dwarfNode(dwarfIndex);
143
144         for (const Edge& edge : graph.graph[currentDwarfNode]) {

```

```

145         if (edge.flow <= 0 || edge.to < firstMineNode || edge.to > lastMineNode)
146         {
147             continue;
148         }
149
150         const size_t mineIndex = static_cast<size_t>(edge.to - firstMineNode);
151         const Krasnal& dwarf = dwarves[dwarfIndex];
152         const Kopalnia& mine = mines[mineIndex];
153         const bool preferred = isPreferredMine(dwarf, mine);
154         const double realDistance = calculateTravelDistance(dwarf, mine);
155
156         result.assignments.emplace_back(dwarf.getId(), mine.getId(), mine.
getResourceType(),
157                                         realDistance, preferred);
158         result.assignedCount++;
159         result.totalDistance += realDistance;
160
161         if (preferred) {
162             result.preferredAssignedCount++;
163         }
164
165         dwarves[dwarfIndex].setAssignedToPreferredResource(preferred);
166         mines[mineIndex].addDwarf(dwarf.getId());
167         assigned[dwarfIndex] = true;
168         break;
169     }
170 }
171
172 for (size_t dwarfIndex = 0; dwarfIndex < dwarves.size(); dwarfIndex++) {
173     if (!assigned[dwarfIndex]) {
174         result.unassignedDwarfIds.push_back(dwarves[dwarfIndex].getId());
175     }
176 }
177
178 WorkAssignmentResult WorkAssignmentSolver::solve() {
179     clearAssignments();
180
181     WorkAssignmentResult result;
182     if (dwarves.empty() || mines.empty()) {
183         for (const Krasnal& dwarf : dwarves) {
184             result.unassignedDwarfIds.push_back(dwarf.getId());
185         }
186         return result;
187     }
188
189     const int vertexCount = sinkNode() + 1;
190     Graph graph(vertexCount);
191     const int preferencePenalty = calculatePreferencePenalty();
192
193     buildGraph(graph, preferencePenalty);
194
195     const int maxFlow = static_cast<int>(dwarves.size());
196     MinCostMaxFlowResult flowResult = min_cost_max_flow(graph, sourceNode(), sinkNode
(), maxFlow);
197     result.optimizationCost = flowResult.cost;
198
199     extractAssignments(graph, result);
200
201     return result;
202 }
203
204 const std::vector<Krasnal>& WorkAssignmentSolver::getDwarves() const {
205     return dwarves;
206 }
207
208 const std::vector<Kopalnia>& WorkAssignmentSolver::getMines() const {
209     return mines;

```

Plik źródłowy 23: /Struktury/WorkAssignmentSolver/WorkAssignmentSolver.cpp

4.3 GuardCommandSolver

Klasa *GuardCommandSolver* wykorzystuje *SegmentTree* do znajdowania najgłośniejszego strażnika na zadanym odcinku granicy. Na wejściu otrzymuje listę strażników w kolejności ustawienia wzdłuż trasy patrolu. Zapytanie *findLoudestGuard(l,r)* zwraca identyfikator strażnika, jego indeks na trasie oraz głośność.

4.3.1 Rozstrzyganie remisów

Samo drzewo przedziałowe zwraca maksimum liczbowe, dlatego w solverze głośność jest kodowana razem z indeksem strażnika. Jeżeli kilku strażników ma taką samą głośność, wybierany jest pierwszy z nich na danym odcinku. Daje to deterministyczny wynik.

4.3.2 Aktualizacja

Metoda *updateLoudness(index, newLoudness)* zmienia głośność strażnika i aktualizuje odpowiadający liść drzewa. Pozwala to reagować na zmianę danych bez przebudowy całej struktury od zera.

4.3.3 Kod źródłowy

```

1  #ifndef GUARD_COMMAND_SOLVER_H
2  #define GUARD_COMMAND_SOLVER_H
3
4  #include "../SegmentTree.h"
5  #include "../Straznik.h"
6
7  #include <memory>
8  #include <vector>
9
10 struct GuardCommandResult {
11     int index;
12     int guardId;
13     int loudness;
14     bool found;
15
16     GuardCommandResult();
17     GuardCommandResult(int index, int guardId, int loudness);
18 };
19
20 class GuardCommandSolver {
21 private:
22     std::vector<Straznik> guards;
23     std::unique_ptr<SegmentTree> loudnessTree;
24
25     std::vector<int> buildEncodedLoudnessValues() const;
26     int encodeValue(size_t index, int loudness) const;
27     int decodeIndex(int encodedValue) const;
28     bool isValidRange(int left, int right) const;
29
30 public:
31     explicit GuardCommandSolver(const std::vector<Straznik>& guards);
32
33     GuardCommandResult findLoudestGuard(int left, int right) const;
34     bool updateLoudness(int index, int newLoudness);
35
36     int size() const;
37     const std::vector<Straznik>& getGuards() const;
38 };
39

```

40 #endif

Plik źródłowy 24: /Struktury/GuardCommandSolver/GuardCommandSolver.h

```
1 #include "GuardCommandSolver.h"
2
3 #include <limits>
4 #include <stdexcept>
5
6 GuardCommandResult::GuardCommandResult()
7     : index(-1), guardId(-1), loudness(0), found(false) {}
8
9 GuardCommandResult::GuardCommandResult(int index, int guardId, int loudness)
10     : index(index), guardId(guardId), loudness(loudness), found(true) {}
11
12 GuardCommandSolver::GuardCommandSolver(const std::vector<Straznik>& guards)
13     : guards(guards), loudnessTree(nullptr) {
14     if (!this->guards.empty()) {
15         loudnessTree.reset(new SegmentTree(buildEncodedLoudnessValues()));
16     }
17 }
18
19 std::vector<int> GuardCommandSolver::buildEncodedLoudnessValues() const {
20     std::vector<int> values;
21     values.reserve(guards.size());
22
23     for (size_t i = 0; i < guards.size(); i++) {
24         values.push_back(encodeValue(i, guards[i].getLoudness()));
25     }
26
27     return values;
28 }
29
30 int GuardCommandSolver::encodeValue(size_t index, int loudness) const {
31     if (loudness < 0 || index >= guards.size()) {
32         throw std::invalid_argument("Invalid guard loudness value");
33     }
34
35     const long long base = static_cast<long long>(guards.size()) + 1;
36     const long long tieBreaker = static_cast<long long>(guards.size()) - static_cast<
37     long long>(index);
38     const long long encoded = static_cast<long long>(loudness) * base + tieBreaker;
39
40     if (encoded > std::numeric_limits<int>::max()) {
41         throw std::overflow_error("Guard loudness value is too large");
42     }
43
44     return static_cast<int>(encoded);
45 }
46
47 int GuardCommandSolver::decodeIndex(int encodedValue) const {
48     if (guards.empty()) {
49         return -1;
50     }
51
52     const int base = static_cast<int>(guards.size()) + 1;
53     const int tieBreaker = encodedValue % base;
54     return static_cast<int>(guards.size()) - tieBreaker;
55 }
56
57 bool GuardCommandSolver::isValidRange(int left, int right) const {
58     return left >= 0 &&
59         right >= left &&
60         right < static_cast<int>(guards.size()) &&
61         loudnessTree != nullptr;
62 }
63
64 GuardCommandResult GuardCommandSolver::findLoudestGuard(int left, int right) const {
```

```

64     if (!isValidRange(left, right)) {
65         return GuardCommandResult();
66     }
67
68     const int encodedResult = loudnessTree->query(left, right);
69     const int index = decodeIndex(encodedResult);
70
71     if (index < left || index > right) {
72         return GuardCommandResult();
73     }
74
75     return GuardCommandResult(index, guards[index].getId(), guards[index].getLoudness());
76 }
77
78 bool GuardCommandSolver::updateLoudness(int index, int newLoudness) {
79     if (index < 0 ||
80         index >= static_cast<int>(guards.size()) ||
81         newLoudness < 0 ||
82         loudnessTree == nullptr) {
83         return false;
84     }
85
86     guards[index].setLoudness(newLoudness);
87     loudnessTree->update(index, encodeValue(static_cast<size_t>(index), newLoudness));
88
89     return true;
90 }
91
92 int GuardCommandSolver::size() const {
93     return static_cast<int>(guards.size());
94 }
95
96 const std::vector<Straznik>& GuardCommandSolver::getGuards() const {
97     return guards;
98 }

```

Plik źródłowy 25: /Struktury/GuardCommandSolver/GuardCommandSolver.cpp

5 Testy funkcji

5.1 test_det.cpp

Sprawdzamy poprawność działania funkcji *det()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1  #include "../det.h"
2  #include<iostream>
3
4  bool test1(){
5      Point p1(0, 0);
6      Point p2(3, 1);
7      Point p3(1, 3);
8
9      const double expected = 8;
10
11     return (det(p1, p2, p3) == expected);
12 }
13
14 bool test2(){
15     Point p1(0, 0);
16     Point p2(3, 1);
17     Point p3(4, -1);
18
19     const double expected = -7;
20
21     return (det(p1, p2, p3) == expected);

```

```

22 }
23
24 bool test3(){
25     Point p1(0, 0);
26     Point p2(3, 1);
27     Point p3(6, 2);
28
29     const double expected = 0;
30
31     return (det(p1, p2, p3) == expected);
32 }
33
34 bool test4(){
35     Point p1(0, 0);
36     Point p2(4, 2);
37     Point p3(2, 1);
38
39     const double expected = 0;
40
41     return (det(p1, p2, p3) == expected);
42 }
43
44 bool test5(){
45     Point p1(0, 0);
46     Point p2(4, 2);
47     Point p3(6, 3);
48
49     const double expected = 0;
50
51     return (det(p1, p2, p3) == expected);
52 }
53
54 bool test6(){
55     Point p1(0, 0);
56     Point p2(4, 2);
57     Point p3(3, 3);
58
59     const double expected = 6;
60
61     return (det(p1, p2, p3) == expected);
62 }
63
64 int main(){
65     std::cout<<"Testy dla funkcji det:"<<std::endl;
66     std::cout<<"Test 1: "<<(test1()? "OK": "ERROR")<<" (det = 8)"<<std::endl;
67     std::cout<<"Test 2: "<<(test2()? "OK": "ERROR")<<" (det = -7)"<<std::endl;
68     std::cout<<"Test 3: "<<(test3()? "OK": "ERROR")<<" (det = 0)"<<std::endl;
69     std::cout<<"Test 4: "<<(test4()? "OK": "ERROR")<<" (det = 0)"<<std::endl;
70     std::cout<<"Test 5: "<<(test5()? "OK": "ERROR")<<" (det = 0)"<<std::endl;
71     std::cout<<"Test 6: "<<(test6()? "OK": "ERROR")<<" (det = 6)"<<std::endl;
72
73     return 0;
74 }

```

Plik źródłowy 26: /Struktury/Funkcje/Testy/test_det.cpp

```

1 #!/bin/bash
2
3 NAME="test_det.exe"
4
5 g++ -Wall -o $NAME test_det.cpp ../det.cpp ../../Point.cpp
6 ./ $NAME
7 rm $NAME

```

Plik źródłowy 27: /Struktury/Funkcje/Testy/run_test_det.sh

5.2 test_is_point_on_segment.cpp

Sprawdzamy poprawność działania funkcji *is_point_on_segment()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```
1 #include <iostream>
2 #include "../is_point_on_segment.cpp"
3
4 bool test1(){
5     Point p1(0,0), p2(4,2), p3(2,1);
6
7     const bool expected = true;
8
9     return (is_point_on_segment(p1,p2,p3) == expected);
10 }
11
12 bool test2(){
13     Point p1(0,0), p2(4,2), p4(6,3);
14
15     const bool expected = false;
16
17     return (is_point_on_segment(p1,p2,p4) == expected);
18 }
19
20 int main(){
21     std::cout<<"Testy dla funkcji is_point_on_segment:"<<std::endl;
22     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (on segment)"<<std::endl;
23     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (not on segment)"<<std::endl;
24
25     return 0;
26 }
```

Plik źródłowy 28: /Struktury/Funkcje/Testy/test_is_point_on_segment.cpp

```
1 #!/bin/bash
2
3 NAME="test_is_point_on_segment.exe"
4
5 g++ -Wall -o $NAME test_is_point_on_segment.cpp ../../Point.cpp ../det.cpp
6 ./ $NAME
7 rm $NAME
```

Plik źródłowy 29: /Struktury/Funkcje/Testy/run_test_is_point_on_segment.sh

5.3 test_do_segments_intersect.cpp

Sprawdzamy poprawność działania funkcji *do_segments_intersect()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```
1 #include <iostream>
2 #include "../do_segments_intersect.cpp"
3
4 // Odcinki rozłącznie, nie przecinają się
5 bool test1(){
6     Point A(1, 1), B(4, 2), C(2, 4), D(5, 5);
7     const bool expected = false;
8
9     return (do_segments_intersect(A, B, C, D) == expected);
10 }
11
12 // Odcinki przecinają się klasycznie X
13 bool test2()
14 {
15     Point A(0,0), B(4,4), C(0,4), D(4,0);
16     return (do_segments_intersect(A,B,C,D) == true);
17 }
18 }
```

```

19
20 // Odcinki stykaja sie w jednym koncowym punkcie
21 bool test3()
22 {
23     Point A(0,0), B(2,2), C(2,2), D(4,0);
24     return(do_segments_intersect(A,B,C,D) == true);
25 }
26
27 // Odcinki wspolliniowe i nachodzace na siebie
28 bool test4()
29 {
30     Point A(0,0), B(4,0), C(2,0), D(6,0);
31     return(do_segments_intersect(A,B,C,D) == true);
32 }
33
34 // Odcinki wspollionowe ale rozlaczne
35 bool test5()
36 {
37     Point A(0,0), B(2,0), C(3,0), D(5,0);
38     return(do_segments_intersect(A,B,C,D) == false);
39 }
40
41 int main(){
42     std::cout<<"Testy dla funkcji do_segments_intersect:"<<std::endl;
43     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (do not intersect)"<<std::endl;
44     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (intersect)"<<std::endl;
45     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (touch at endpoint)"<<std::endl;
46     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (collinear overlapping)"<<std::endl;
47     // wspollionowe nachodzace
48     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (collinear disjoining)"<<std::endl;
49     // wspolliniowe rozlaczne
50     return 0;
}

```

Plik źródłowy 30: /Struktury/Funkcje/Testy/test_do_segments_intersect.cpp

```

1 #!/bin/bash
2
3 NAME="test_do_segments_intersect.exe"
4
5 g++ -Wall -o $NAME test_do_segments_intersect.cpp ../../Point.cpp ../../det.cpp ../../
6     is_point_on_segment.cpp
7 ./ $NAME
8 rm $NAME

```

Plik źródłowy 31: /Struktury/Funkcje/Testy/run_test_do_segments_intersect.sh

5.4 test_is_point_in_polygon.cpp

Sprawdzamy poprawność działania funkcji *is_point_in_polygon()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1 #include "../is_point_in_polygon.h"
2 #include<iostream>
3
4 bool test1(){
5     std::vector<Point> polygon = {Point(0, 0), Point(5, 1), Point(3, 3), Point(5, 5),
6     Point(2, 4), Point(0, 5)};
7     Point p = Point(3, 2);
8
9     const bool expected = true;
10
11     return (is_point_in_polygon(p, polygon) == expected);
12 }
13
14 bool test2(){

```

```

14     std::vector<Point> polygon = {Point(0, 0), Point(5, 1), Point(3, 3), Point(5, 5),
15         Point(2, 4), Point(0, 5)};
16     Point p = Point(4, 3);
17
18     const bool expected = false;
19
20     return (is_point_in_polygon(p, polygon) == expected);
21 }
22
23 int main(){
24     std::cout<<"Testy dla funkcji is_point_in_polygon:"<<std::endl;
25     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (in polygon)"<<std::endl;
26     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (not in polygon)"<<std::endl;
27
28     return 0;
29 }

```

Plik źródłowy 32: /Struktury/Funkcje/Testy/test_is_point_in_polygon.cpp

```

1  #!/bin/bash
2
3  NAME="test_is_point_in_polygon.exe"
4
5  g++ -Wall -o $NAME test_is_point_in_polygon.cpp ../is_point_in_polygon.cpp ../../
6  Point.cpp ../is_point_on_segment.cpp ../det.cpp ../do_segments_intersect.cpp
7  ./ $NAME
8  rm $NAME

```

Plik źródłowy 33: /Struktury/Funkcje/Testy/run_test_is_point_in_polygon.sh

5.5 test_graham_scan.cpp

Sprawdzamy poprawność działania funkcji *graham_scan()* wywołując kilka razy tę funkcję i sprawdzania czy wynik jej jest poprawny z oczekiwanym

```

1  #include "../graham_scan.h"
2  #include "../distance.h"
3  #include <iostream>
4
5  // Test 1 - pusyt zbior -> otoczka tez pusta
6  bool test1(){
7      std::vector<Point> points, expected;
8
9      points = {};
10
11      expected = {};
12      points = graham_scan(points);
13
14      if(points.size() != expected.size()) return false;
15
16      for(unsigned int i = 0; i < points.size(); i++){
17          if(points[i] != expected[i]) return false;
18      }
19
20      return true;
21 }
22
23 // Test 2 - jeden punkt
24 bool test2(){
25     std::vector<Point> points, expected;
26
27     points = {Point(0, 0)};
28
29     expected = points;
30     points = graham_scan(points);
31
32     if(points.size() != expected.size()) return false;

```

```

33
34     for(unsigned int i = 0; i < points.size(); i++){
35         if(points[i] != expected[i]) return false;
36     }
37
38     return true;
39 }
40
41 // Test 3 - dwa identyczne pkt -> jeden w otoczce
42 bool test3(){
43     std::vector<Point> points, expected;
44
45     points = {Point(0, 0), Point(0, 0)};
46
47     expected = {Point(0, 0)};
48     points = graham_scan(points);
49
50     if(points.size() != expected.size()) return false;
51
52     for(unsigned int i = 0; i < points.size(); i++){
53         if(points[i] != expected[i]) return false;
54     }
55
56     return true;
57 }
58
59 // Test 4 dwa rozne pkt + duplikat -> tylko unikalne pkt
60 bool test4(){
61     std::vector<Point> points, expected;
62
63     points = {Point(0, 0), Point(0, 0), Point(1, 2)};
64
65     expected = {Point(0, 0), Point(1, 2)};
66     points = graham_scan(points);
67
68     if(points.size() != expected.size()) return false;
69
70     for(unsigned int i = 0; i < points.size(); i++){
71         if(points[i] != expected[i]) return false;
72     }
73
74     return true;
75 }
76
77 // Test 5 - sprawdzenie poprawnego wyboru pkt
78 bool test5(){
79     std::vector<Point> points, expected;
80
81     points = {Point(0, 0), Point(1, 1), Point(2, 0), Point(3, 0)};
82
83     expected = {Point(0, 0), Point(3, 0), Point(1, 1)};
84     points = graham_scan(points);
85
86     if(points.size() != expected.size()) return false;
87
88     for(unsigned int i = 0; i < points.size(); i++){
89         if(points[i] != expected[i]) return false;
90     }
91
92     return true;
93 }
94
95 // Test 6 - poprawnosc pkt (wiekszy zbior)
96 bool test6(){
97     std::vector<Point> points, expected;
98
99     points = {
100         Point(0, 0), Point(1, 0), Point(2, 0), Point(2, 1), Point(4, 2),

```

```

101     Point(3, 4), Point(1, 3), Point(5, 1), Point(3, 0), Point(2, 2)
102 };
103
104     expected = {Point(0, 0), Point(3, 0), Point(5, 1), Point(3, 4), Point(1, 3)};
105     points = graham_scan(points);
106
107     if(points.size() != expected.size()) return false;
108
109     for(unsigned int i = 0; i < points.size(); i++){
110         if(points[i] != expected[i]) return false;
111     }
112
113     return true;
114 }
115
116 // Test 7 - punkty wspolliniowe -> tylko skrajne powinny zostac
117 bool test7()
118 {
119     std::vector<Point> points, expected;
120
121     points = {
122         Point(0,0), Point(1,1), Point(2,2),
123         Point(3,3), Point(4,4)
124     };
125
126     expected = {Point(0,0), Point(4,4)};
127     points = graham_scan(points);
128
129     if(points.size() != expected.size()) return false;
130
131     for(unsigned int i = 0; i < points.size(); i++){
132         if(points[i] != expected[i]) return false;
133     }
134
135     return true;
136 }
137
138 int main(){
139     std::cout<<"Testy dla funkcji graham_scan:"<<std::endl;
140     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (empty set)"<<std::endl;
141     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (1 element set)"<<std::endl;
142     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (2 element set)"<<std::endl;
143     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (3 element set)"<<std::endl;
144     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (4 element set)"<<std::endl;
145     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (10 element set)"<<std::endl;
146     std::cout<<"Test 7: "<<(test7()?"OK":"ERROR")<<" (collinear points)"<<std::endl;
147
148     return 0;
149 }
150

```

Plik źródłowy 34: /Struktury/Funkcje/Testy/test_graham_scan.cpp

```

1 #!/bin/bash
2
3 NAME="test_graham_scan.exe"
4
5 g++ -Wall -o $NAME test_graham_scan.cpp ../graham_scan.cpp ../../Point.cpp ../det.cpp
6     ../distance.cpp
7 ./ $NAME
8 rm $NAME

```

Plik źródłowy 35: /Struktury/Funkcje/Testy/run_test_graham_scan.sh

6 Testy struktur

6.1 test_point.cpp

Sprawdzamy poprawność struktury *Point* poprzez sprawdzenie czy określone operatory poprawnie działają i wykonują to co chcemy

```
1 #include <iostream>
2 #include <sstream>
3 #include "../Point.h"
4
5 bool test1(){
6     Point A(1,2);
7     Point B(4,5);
8
9     const bool expected = false;
10
11     return ((A > B) == expected);
12 }
13
14 bool test2(){
15     Point A(1,2);
16     Point B(4,5);
17
18     const bool expected = true;
19
20     return ((A < B) == expected);
21 }
22
23 bool test3(){
24     Point A(1, 2);
25
26     try{
27         std::ostringstream str;
28         str<<A;
29         return true;
30     } catch(...){
31         return false;
32     }
33 }
34
35 bool test4(){
36     Point A(1, 2);
37     Point B(1, 2);
38
39     const bool expected = true;
40
41     return ((A == B) == expected);
42 }
43
44 bool test5(){
45     Point A(1, 2);
46     Point B(3, 4);
47
48     const bool expected = true;
49
50     return ((A != B) == expected);
51 }
52
53 bool test6(){
54     Point A(1, 2);
55     Point B(1, 2);
56
57     const bool expected = true;
58
59     return ((A <= B) == expected);
60 }
61
```

```

62 bool test7(){
63     Point A(1, 2);
64     Point B(0, 0);
65
66     const bool expected = true;
67
68     return ((A >= B) == expected);
69 }
70
71 bool test8(){
72     Point A(1, 2);
73     Point B(3, 4);
74     B = A;
75
76     const bool expected = true;
77
78     return ((A == B) == expected);
79 }
80
81 int main(){
82     std::cout<<"Testy dla klasy Point:"<<std::endl;
83     std::cout<<"Test 1: "<<(test1())?"OK":"ERROR"<<" (operator >)"<<std::endl;
84     std::cout<<"Test 2: "<<(test2())?"OK":"ERROR"<<" (operator <)"<<std::endl;
85     std::cout<<"Test 3: "<<(test3())?"OK":"ERROR"<<" (operator <=)"<<std::endl;
86     std::cout<<"Test 4: "<<(test4())?"OK":"ERROR"<<" (operator ==)"<<std::endl;
87     std::cout<<"Test 5: "<<(test5())?"OK":"ERROR"<<" (operator !=)"<<std::endl;
88     std::cout<<"Test 6: "<<(test6())?"OK":"ERROR"<<" (operator <=)"<<std::endl;
89     std::cout<<"Test 7: "<<(test7())?"OK":"ERROR"<<" (operator >=)"<<std::endl;
90     std::cout<<"Test 8: "<<(test8())?"OK":"ERROR"<<" (operator ==)"<<std::endl;
91
92     return 0;
93 }

```

Plik źródłowy 36: /Struktury/Funkcje/Testy/test_point.cpp

```

1 #!/bin/bash
2
3 NAME="test_point.exe"
4
5 g++ -Wall -o $NAME test_point.cpp ../../Point.cpp
6 ./ $NAME
7 rm $NAME

```

Plik źródłowy 37: /Struktury/Funkcje/Testy/run_test_point.sh

6.2 test_huffman.cpp

Sprawdzamy poprawność zbioru funkcji *Huffman* poprzez sprawdzenie czy funkcja *Huffman* zwraca poprawne wyniki (przy okazji sprawdzamy wszystkie zdefiniowane funkcje pomocnicze)

```

1 #include "../Huffman.h"
2 #include<iostream>
3 using namespace std;
4
5 bool test1(){
6     vector<char> characters = {'a', 'b', 'c', 'd', 'e', 'f'};
7     vector<int> frequencies = {45, 13, 12, 16, 9, 5};
8
9     vector<HuffmanCode> expectedCodes = {
10         {'a', "0"},
11         {'b', "101"},
12         {'c', "100"},
13         {'d', "111"},
14         {'e', "1101"},
15         {'f', "1100"}
16     };

```

```

17     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
18
19
20     return HuffmanEquals(huffmanCodes, expectedCodes);
21 }
22
23 bool test2(){
24     vector<char> characters = {'a', 'b', 'c', 'd', 'e', 'f'};
25     vector<int> frequencies = {1, 1, 1, 1, 1, 1};
26
27     vector<HuffmanCode> expectedCodes = {
28         {'a', "110"},
29         {'b', "00"},
30         {'c', "111"},
31         {'d', "01"},
32         {'e', "101"},
33         {'f', "100"}
34     };
35
36     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
37
38     return HuffmanEquals(huffmanCodes, expectedCodes);
39 }
40
41 bool test3(){
42     vector<char> characters = {'m', 'k', 'x', 'y', 'z'};
43     vector<int> frequencies = {18, 9, 2, 3, 7};
44
45     vector<HuffmanCode> expectedCodes = {
46         {'k', "10"},
47         {'m', "0"},
48         {'x', "1100"},
49         {'y', "1101"},
50         {'z', "111"}
51     };
52
53     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
54
55     return HuffmanEquals(huffmanCodes, expectedCodes);
56 }
57
58 bool test4(){
59     vector<char> characters = {};
60     vector<int> frequencies = {};
61
62     vector<HuffmanCode> expectedCodes = {};
63
64     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
65
66     return HuffmanEquals(huffmanCodes, expectedCodes);
67 }
68
69 bool test5(){
70     vector<char> characters = {'x', 'y', 'z'};
71     vector<int> frequencies = {};
72
73     vector<HuffmanCode> expectedCodes = {};
74
75     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
76
77     return HuffmanEquals(huffmanCodes, expectedCodes);
78 }
79
80 bool test6(){
81     vector<char> characters = {};
82     vector<int> frequencies = {2, 3, 5, 7};
83
84     vector<HuffmanCode> expectedCodes = {};

```

```

85     vector<HuffmanCode> huffmanCodes = HuffmanSort(Huffman(characters, frequencies));
86
87
88     return HuffmanEquals(huffmanCodes, expectedCodes);
89 }
90
91 int main(){
92     std::cout<<"Testy dla funkcji huffman:"<<std::endl;
93     std::cout<<"Test 1: "<<(test1()?"OK":"ERROR")<<" (normal set)"<<std::endl;
94     std::cout<<"Test 2: "<<(test2()?"OK":"ERROR")<<" (same freq)"<<std::endl;
95     std::cout<<"Test 3: "<<(test3()?"OK":"ERROR")<<" (normal set)"<<std::endl;
96     std::cout<<"Test 4: "<<(test4()?"OK":"ERROR")<<" (both empty)"<<std::endl;
97     std::cout<<"Test 5: "<<(test5()?"OK":"ERROR")<<" (freq empty)"<<std::endl;
98     std::cout<<"Test 6: "<<(test6()?"OK":"ERROR")<<" (chars empty)"<<std::endl;
99
100     return 0;
101 }

```

Plik źródłowy 38: /Struktury/Funkcje/Testy/test_huffman.cpp

```

1 #!/bin/bash
2
3 NAME="test_huffman.exe"
4
5 g++ -Wall -o $NAME test_huffman.cpp ../../Huffman.cpp
6 ./ $NAME
7 rm $NAME

```

Plik źródłowy 39: /Struktury/Funkcje/Testy/run_test_huffman.sh

6.3 test_segment_tree

Sprawdzamy poprawność struktury *SegmentTree* poprzez sprawdzenie czy funkcje *query* oraz *update* zwracają poprawne wyniki.

```

1 #include "../../SegmentTree.h"
2 #include <iostream>
3
4 // Test 1: Przykład z wykładu (indeksowanie od 0)
5 bool test1()
6 {
7     SegmentTree t1({5, 2, 7, 1, 3, 6, 4, 8});
8     const int expected = 8;
9     return (t1.query(2, 7) == expected);
10 }
11
12 // Test 2
13 bool test2()
14 {
15     SegmentTree t2({2, 3, -1, 5, -2, 4, 8, 10});
16     const int expected = 5;
17     return (t2.query(2, 4) == expected);
18 }
19
20 // Test 3: Same wartosci ujemne (sprawdzenie czy poprawnie zwraca wartosci ponizej 0)
21 bool test3()
22 {
23     SegmentTree t3({-10, -5, -20, -3, -15});
24     const int expected = -3;
25     return (t3.query(0, 4) == expected);
26 }
27
28 // Test 4: Zapytanie o przedzial jednoelementowy (L == R)
29 bool test4()
30 {
31     SegmentTree t4({1, 5, 9, 2});
32     const int expected = 9;

```

```

33     return (t4.query(2, 2) == expected);
34 }
35
36 // Test 5: Test operacji UPDATE (mala wartosc na olbrzymia)
37 bool test5()
38 {
39     SegmentTree t5({1, 2, 3, 4});
40     t5.update(1, 100);
41     const int expected = 100;
42     return (t5.query(0, 3) == expected);
43 }
44
45 // Test 6: Test operacji UPDATE, ktora zmniejsza maksimum
46 bool test6()
47 {
48     SegmentTree t6({5, 20, 10});
49     t6.update(1, 2);
50     const int expected = 10;
51     return (t6.query(0, 2) == expected);
52 }
53
54 int main() {
55     std::cout << "Testy dla drzewa przedzialowego (MaxQuery):" << std::endl;
56     std::cout << "Test 1: " << (test1() ? "OK" : "ERROR") << " (Expected: 8)" << std::endl;
57     std::cout << "Test 2: " << (test2() ? "OK" : "ERROR") << " (Expected: 5)" << std::endl;
58     std::cout << "Test 3: " << (test3() ? "OK" : "ERROR") << " (Expected: -3)" << std::endl;
59     std::cout << "Test 4: " << (test4() ? "OK" : "ERROR") << " (Expected: 9)" << std::endl;
60     std::cout << "Test 5: " << (test5() ? "OK" : "ERROR") << " (Expected: 100 po update)" << std::endl;
61     std::cout << "Test 6: " << (test6() ? "OK" : "ERROR") << " (Expected: 10 po zmniejszeniu max)" << std::endl;
62
63     return 0;
64 }

```

Plik źródłowy 40: /Struktury/Funkcje/Testy/test_segment_tree.cpp

6.4 test_class_BorderPatrol.cpp

Testy sprawdzają zachowanie *BorderPatrolSolver* dla pustego zbioru kopalni, jednej kopalni, dwóch kopalni, kwadratu oraz punktu znajdującego się wewnątrz otoczki. Dzięki temu weryfikowana jest zarówno długość patrolu, jak i poprawność wyznaczonej otoczki.

```

1 #include "../BorderPatrolSolver.h"
2 #include "../EPS.h"
3 #include <iostream>
4
5 // Brak kopalni - odległość powinna być 0
6 bool test1() {
7     std::vector<Point> mines = {};
8     BorderPatrolSolver solver(mines);
9
10    double dist = solver.calculatePatrolDistance();
11    return cmpDouble(dist, 0.0) == 0.0;
12 }
13
14 // Jedna kopalnia - nie ma trasy do okrążenia
15 bool test2() {
16     std::vector<Point> mines = {Point(1, 1)};
17     BorderPatrolSolver solver(mines);
18
19    double dist = solver.calculatePatrolDistance();
20    return cmpDouble(dist, 0.0) == 0.0;

```

```

21 }
22
23 // Dwie kopalnie - trasa to odcinek tam i z powrotem
24 bool test3() {
25     std::vector<Point> mines = {Point(0, 0), Point(3, 0)};
26     BorderPatrolSolver solver(mines);
27
28     double dist = solver.calculatePatrolDistance();
29     return cmpDouble(dist, 6.0) == 0;
30 }
31
32 // Cztery kopalnie tworzą kwadrat 4x4 - obwód = 16
33 bool test4() {
34     std::vector<Point> mines = {
35         Point(0, 0), Point(4, 0), Point(4, 4), Point(0, 4)
36     };
37     BorderPatrolSolver solver(mines);
38
39     double dist = solver.calculatePatrolDistance();
40     return cmpDouble(dist, 16.0) == 0;
41 }
42
43 // Jeden punkt wewnątrz kwadratu - nie wpływa na otoczkę ani obwód
44 bool test5() {
45     std::vector<Point> mines = {
46         Point(0, 0), Point(4, 0), Point(4, 4), Point(0, 4), Point(2, 2)
47     };
48     BorderPatrolSolver solver(mines);
49
50     double dist = solver.calculatePatrolDistance();
51     return cmpDouble(dist, 16.0) == 0;
52 }
53
54 // Sprawdzamy czy calculateConvexHull zwraca właściwe punkty
55 bool test6() {
56     std::vector<Point> mines = {
57         Point(0, 0), Point(4, 0), Point(4, 4), Point(0, 4), Point(2, 2)
58     };
59     BorderPatrolSolver solver(mines);
60     std::vector<Point> hull = solver.calculateConvexHull();
61
62     // (2,2) nie powinien być w otoczce
63     for(const Point& p : hull)
64         if(p == Point(2, 2)) return false;
65     return hull.size() == 4;
66 }
67
68 int main() {
69     std::cout << "Testy dla BorderPatrolSolver:" << std::endl;
70
71     std::cout << "Test 1: " << (test1() ? "OK" : "ERROR") << " (pusty zbior)" << std::endl;
72     std::cout << "Test 2: " << (test2() ? "OK" : "ERROR") << " (jedna kopalnia)" << std::endl;
73     std::cout << "Test 3: " << (test3() ? "OK" : "ERROR") << " (dwie kopalnie)" << std::endl;
74     std::cout << "Test 4: " << (test4() ? "OK" : "ERROR") << " (kwadrat)" << std::endl;
75     std::cout << "Test 5: " << (test5() ? "OK" : "ERROR") << " (punkt wewnątrz)" << std::endl;
76     std::cout << "Test 6: " << (test6() ? "OK" : "ERROR") << " (poprawnosc otoczki)" << std::endl;
77
78     return 0;
79 }

```

Plik źródłowy 41: /Struktury/Funkcje/Testy/test_class_BorderPatrol.cpp

6.5 test_work_assignment_solver.cpp

Testy *WorkAssignmentSolver* sprawdzają prosty przydział jeden do jednego, priorytet preferowanego surowca względem krótszej drogi, obsługę pojemności kopalni oraz sytuację, w której krasnal nie ma możliwego przydziału.

```
1 #include "../WorkAssignmentSolver.h"
2
3 #include <cmath>
4 #include <iostream>
5 #include <vector>
6
7 bool nearDouble(double a, double b) {
8     return std::fabs(a - b) < 0.000001;
9 }
10
11 bool testSimpleAssignment() {
12     std::vector<Krasnal> dwarves = {
13         Krasnal(1, Point(0, 0), {"zloto"}, "zloto", false)
14     };
15     std::vector<Kopalnia> mines = {
16         Kopalnia(10, Point(3, 4), "zloto", 1, {})
17     };
18
19     WorkAssignmentSolver solver(dwarves, mines);
20     WorkAssignmentResult result = solver.solve();
21
22     return result.assignedCount == 1 &&
23         result.preferredAssignedCount == 1 &&
24         result.unassignedDwarfIds.empty() &&
25         result.assignments.size() == 1 &&
26         result.assignments[0].dwarfId == 1 &&
27         result.assignments[0].mineId == 10 &&
28         nearDouble(result.totalDistance, 5.0) &&
29         solver.getMines()[0].getAssignedDwarves().size() == 1;
30 }
31
32 bool testPreferenceBeforeDistance() {
33     std::vector<Krasnal> dwarves = {
34         Krasnal(1, Point(0, 0), {"zloto", "miedz"}, "zloto", false)
35     };
36     std::vector<Kopalnia> mines = {
37         Kopalnia(10, Point(10, 0), "zloto", 1, {}),
38         Kopalnia(11, Point(1, 0), "miedz", 1, {})
39     };
40
41     WorkAssignmentSolver solver(dwarves, mines);
42     WorkAssignmentResult result = solver.solve();
43
44     return result.assignedCount == 1 &&
45         result.preferredAssignedCount == 1 &&
46         result.assignments.size() == 1 &&
47         result.assignments[0].mineId == 10;
48 }
49
50 bool testMineCapacity() {
51     std::vector<Krasnal> dwarves = {
52         Krasnal(1, Point(0, 0), {"wegiel"}, "wegiel", false),
53         Krasnal(2, Point(1, 0), {"wegiel"}, "wegiel", false)
54     };
55     std::vector<Kopalnia> mines = {
56         Kopalnia(10, Point(0, 1), "wegiel", 2, {})
57     };
58
59     WorkAssignmentSolver solver(dwarves, mines);
60     WorkAssignmentResult result = solver.solve();
61
62     return result.assignedCount == 2 &&
```

```

63         result.preferredAssignedCount == 2 &&
64         result.unassignedDwarfIds.empty() &&
65         solver.getMines()[0].getAssignedDwarves().size() == 2;
66     }
67
68     bool testUnassignedDwarf() {
69         std::vector<Krasnal> dwarves = {
70             Krasnal(1, Point(0, 0), {"wegiel"}, "wegiel", false)
71         };
72         std::vector<Kopalnia> mines = {
73             Kopalnia(10, Point(0, 1), "zloto", 1, {})
74         };
75
76         WorkAssignmentSolver solver(dwarves, mines);
77         WorkAssignmentResult result = solver.solve();
78
79         return result.assignedCount == 0 &&
80                result.preferredAssignedCount == 0 &&
81                result.unassignedDwarfIds.size() == 1 &&
82                result.unassignedDwarfIds[0] == 1 &&
83                result.assignments.empty();
84     }
85
86     int main() {
87         const bool test1 = testSimpleAssignment();
88         const bool test2 = testPreferenceBeforeDistance();
89         const bool test3 = testMineCapacity();
90         const bool test4 = testUnassignedDwarf();
91
92         std::cout << "Testy dla WorkAssignmentSolver:" << std::endl;
93         std::cout << "Test 1: " << (test1 ? "OK" : "ERROR") << " (prosty przydzial)" <<
94             std::endl;
95         std::cout << "Test 2: " << (test2 ? "OK" : "ERROR") << " (preferencja przed
96             dystansem)" << std::endl;
97         std::cout << "Test 3: " << (test3 ? "OK" : "ERROR") << " (pojemnosc kopalni)" <<
98             std::endl;
99         std::cout << "Test 4: " << (test4 ? "OK" : "ERROR") << " (brak mozliwego
100             przydzialu)" << std::endl;
101
102         return (test1 && test2 && test3 && test4) ? 0 : 1;
103     }

```

Plik źródłowy 42: /Struktury/WorkAssignmentSolver/Testy/test_work_assignment_solver.cpp

6.6 test_segment_tree.cpp

Testy drzewa przedziałowego sprawdzają zapytania o maksimum na różnych zakresach, obsługę wartości ujemnych, zapytanie jednoelementowe oraz aktualizacje zwiększające i zmniejszające maksimum.

```

1  #include "../SegmentTree.h"
2  #include <iostream>
3
4  // Test 1: Przykład z wykładu (indeksowanie od 0)
5  bool test1()
6  {
7      SegmentTree t1({5, 2, 7, 1, 3, 6, 4, 8});
8      const int expected = 8;
9      return (t1.query(2, 7) == expected);
10 }
11
12 // Test 2
13 bool test2()
14 {
15     SegmentTree t2({2, 3, -1, 5, -2, 4, 8, 10});
16     const int expected = 5;
17     return (t2.query(2, 4) == expected);

```

```

18 }
19
20 // Test 3: Same wartosci ujemne (sprawdzenie czy poprawnie zwraca wartosci ponizej 0)
21 bool test3()
22 {
23     SegmentTree t3({-10, -5, -20, -3, -15});
24     const int expected = -3;
25     return (t3.query(0, 4) == expected);
26 }
27
28 // Test 4: Zapytanie o przedzial jednoelementowy (L == R)
29 bool test4()
30 {
31     SegmentTree t4({1, 5, 9, 2});
32     const int expected = 9;
33     return (t4.query(2, 2) == expected);
34 }
35
36 // Test 5: Test operacji UPDATE (mala wartosc na olbrzymia)
37 bool test5()
38 {
39     SegmentTree t5({1, 2, 3, 4});
40     t5.update(1, 100);
41     const int expected = 100;
42     return (t5.query(0, 3) == expected);
43 }
44
45 // Test 6: Test operacji UPDATE, ktora zmniejsza maksimum
46 bool test6()
47 {
48     SegmentTree t6({5, 20, 10});
49     t6.update(1, 2);
50     const int expected = 10;
51     return (t6.query(0, 2) == expected);
52 }
53
54 int main() {
55     std::cout << "Testy dla drzewa przedzialowego (MaxQuery):" << std::endl;
56     std::cout << "Test 1: " << (test1() ? "OK" : "ERROR") << " (Expected: 8)" << std::endl;
57     std::cout << "Test 2: " << (test2() ? "OK" : "ERROR") << " (Expected: 5)" << std::endl;
58     std::cout << "Test 3: " << (test3() ? "OK" : "ERROR") << " (Expected: -3)" << std::endl;
59     std::cout << "Test 4: " << (test4() ? "OK" : "ERROR") << " (Expected: 9)" << std::endl;
60     std::cout << "Test 5: " << (test5() ? "OK" : "ERROR") << " (Expected: 100 po update)" << std::endl;
61     std::cout << "Test 6: " << (test6() ? "OK" : "ERROR") << " (Expected: 10 po zmniejszeniu max)" << std::endl;
62
63     return 0;
64 }

```

Plik źródłowy 43: /Struktury/Funkcje/Testy/test_segment_tree.cpp

6.7 test_guard_command_solver.cpp

Testy *GuardCommandSolver* sprawdzają wybór najgłośniejszego strażnika na całym zakresie, na podzakresie, rozstrzyganie remisu przez wybór pierwszego strażnika, aktualizację głośności, błędny zakres oraz pustą listę strażników.

```

1 #include "../GuardCommandSolver.h"
2
3 #include <iostream>
4 #include <vector>
5

```

```

6  std::vector<Straznik> createGuards() {
7      return {
8          Straznik(100, 14),
9          Straznik(101, 9),
10         Straznik(102, 18),
11         Straznik(103, 13),
12         Straznik(104, 18),
13         Straznik(105, 11)
14     };
15 }
16
17 bool testWholeRange() {
18     GuardCommandSolver solver(createGuards());
19     GuardCommandResult result = solver.findLoudestGuard(0, solver.size() - 1);
20
21     return result.found &&
22            result.index == 2 &&
23            result.guardId == 102 &&
24            result.loudness == 18;
25 }
26
27 bool testSubRange() {
28     GuardCommandSolver solver(createGuards());
29     GuardCommandResult result = solver.findLoudestGuard(3, 5);
30
31     return result.found &&
32            result.index == 4 &&
33            result.guardId == 104 &&
34            result.loudness == 18;
35 }
36
37 bool testTieReturnsFirstGuard() {
38     GuardCommandSolver solver({
39         Straznik(1, 7),
40         Straznik(2, 7),
41         Straznik(3, 5)
42     });
43
44     GuardCommandResult result = solver.findLoudestGuard(0, 2);
45
46     return result.found &&
47            result.index == 0 &&
48            result.guardId == 1 &&
49            result.loudness == 7;
50 }
51
52 bool testUpdateLoudness() {
53     GuardCommandSolver solver(createGuards());
54
55     if (!solver.updateLoudness(1, 20)) {
56         return false;
57     }
58
59     GuardCommandResult result = solver.findLoudestGuard(0, solver.size() - 1);
60
61     return result.found &&
62            result.index == 1 &&
63            result.guardId == 101 &&
64            result.loudness == 20;
65 }
66
67 bool testInvalidRange() {
68     GuardCommandSolver solver(createGuards());
69     GuardCommandResult result = solver.findLoudestGuard(4, 2);
70
71     return !result.found;
72 }
73

```

```

74 bool testEmptyGuards() {
75     GuardCommandSolver solver({});
76     GuardCommandResult result = solver.findLoudestGuard(0, 0);
77
78     return solver.size() == 0 && !result.found;
79 }
80
81 int main() {
82     const bool test1 = testWholeRange();
83     const bool test2 = testSubRange();
84     const bool test3 = testTieReturnsFirstGuard();
85     const bool test4 = testUpdateLoudness();
86     const bool test5 = testInvalidRange();
87     const bool test6 = testEmptyGuards();
88
89     std::cout << "Testy dla GuardCommandSolver:" << std::endl;
90     std::cout << "Test 1: " << (test1 ? "OK" : "ERROR") << " (caly zakres)" << std::endl;
91     std::cout << "Test 2: " << (test2 ? "OK" : "ERROR") << " (podzakres)" << std::endl;
92     std::cout << "Test 3: " << (test3 ? "OK" : "ERROR") << " (remis wybiera pierwszego)" << std::endl;
93     std::cout << "Test 4: " << (test4 ? "OK" : "ERROR") << " (aktualizacja glosnosci)" << std::endl;
94     std::cout << "Test 5: " << (test5 ? "OK" : "ERROR") << " (bledny zakres)" << std::endl;
95     std::cout << "Test 6: " << (test6 ? "OK" : "ERROR") << " (brak straznikow)" << std::endl;
96
97     return (test1 && test2 && test3 && test4 && test5 && test6) ? 0 : 1;
98 }

```

Plik źródłowy 44: /Struktury/GuardCommandSolver/Testy/test_guard_command_solver.cpp

7 Prezentacja działania

7.1 Główny przepływ programu

Główny program łączy przygotowane moduły w jeden przepływ danych:

1. *DataLoader* wczytuje krasnale, kopalnie i strażników z pliku wejściowego.
2. *WorkAssignmentSolver* przydziela krasnale do kopalni za pomocą minimalnego kosztu maksymalnego przepływu.
3. Z wyniku przydziału wybierane są tylko aktywne kopalnie, czyli te, do których trafił przynajmniej jeden krasnal.
4. *BorderPatrolSolver* wyznacza otoczkę wypukłą aktywnych kopalni i długość trasy patrolu.
5. *GuardCommandSolver* odpowiada na zapytania o najgłośniejszego strażnika na wskazanym odcinku granicy.

7.2 Przykładowy wynik dla Dane/dane.txt

Dla przykładowych danych wejściowych program przydziela wszystkie trzy krasnale do kopalni zgodnych z ich preferencjami. Aktywne są kopalnie 10, 11 oraz 12. Długość trasy patrolu po ich otoczce wynosi około 19.23.

```

1 Krasnale: 3
2 Kopalnie: 4
3 Straznicy: 6
4

```

```

5 Przydział pracy:
6 Krasnal 1 -> kopalnia 10 (zloto), dystans: 2.83 [preferowany surowiec]
7 Krasnal 2 -> kopalnia 11 (wegiel), dystans: 1.41 [preferowany surowiec]
8 Krasnal 3 -> kopalnia 12 (miedz), dystans: 2.24 [preferowany surowiec]
9
10 Przydzielono: 3/3
11 Przydziały preferowane: 3
12 Łączny dystans: 6.48
13
14 Aktywne kopalnie:
15 Kopalnia 10 (zloto) (2.00, 2.00), krasnale: 1
16 Kopalnia 11 (wegiel) (9.00, 2.00), krasnale: 2
17 Kopalnia 12 (miedz) (5.00, 7.00), krasnale: 3
18
19 Punkty trasy patrolu:
20 (2.00, 2.00)
21 (9.00, 2.00)
22 (5.00, 7.00)
23 Długość trasy patrolu: 19.23
24
25 Obrona granicy:
26 Najgłośniejszy na całej trasie: strażnik 102 na pozycji 2, głośność: 18
27 Odcinek ataku [1, 3]: strażnik 102 na pozycji 2, głośność: 18

```

Plik źródłowy 45: Przykładowe wyjście programu